RV 32I Multi cycle APB BUS 및 Peripheral

7조
나연우, 최여지, 우상욱

목 차

01

CPU

- Block diagram
- FSM
- Single vs Multi
- Multi cycle

02

Bus

- Protocol
- Memory Map
- Block diagram
- FSM
- Simulation

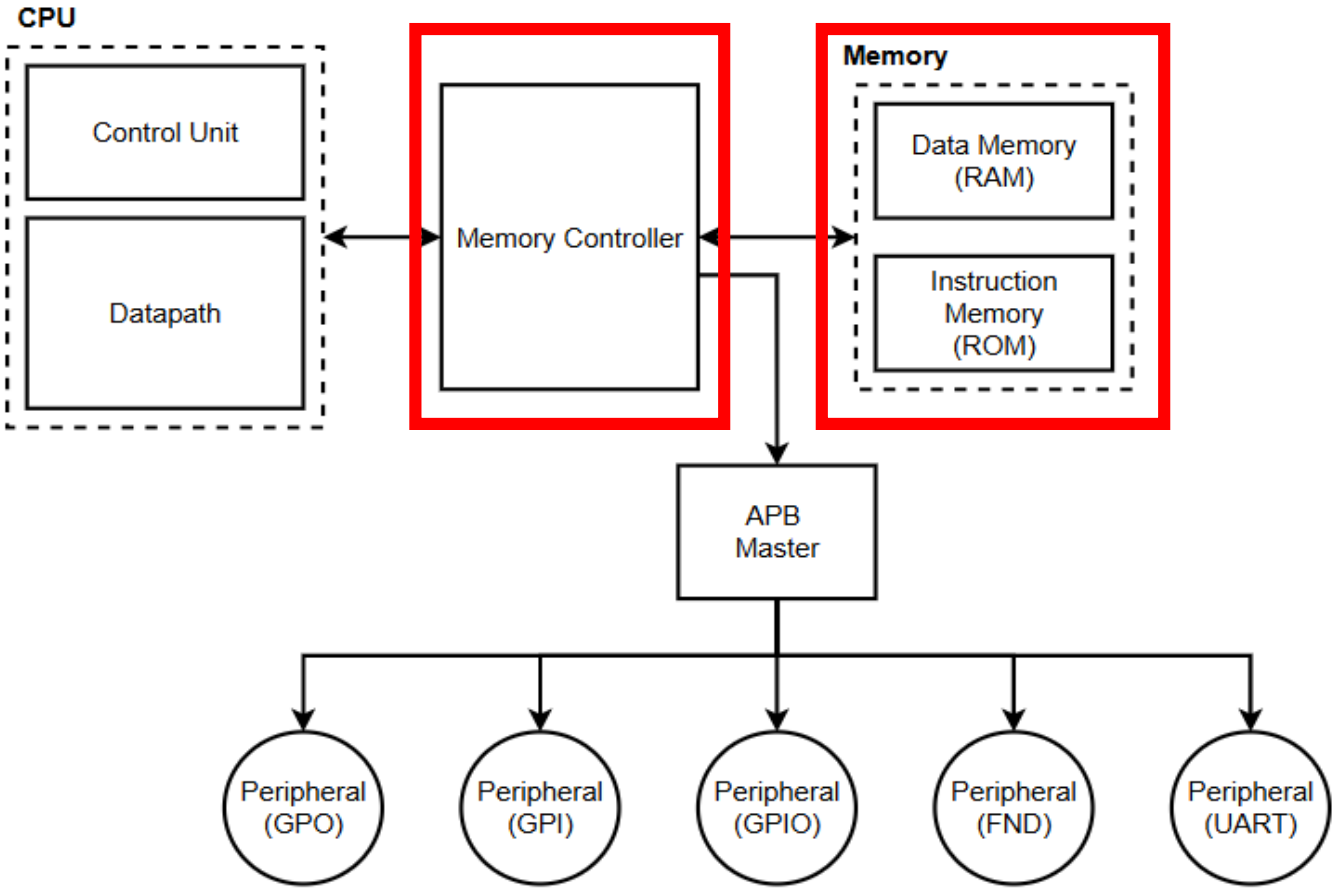
03

Peripheral

- GPO
- GPI
- GPIO
- FND
- UART

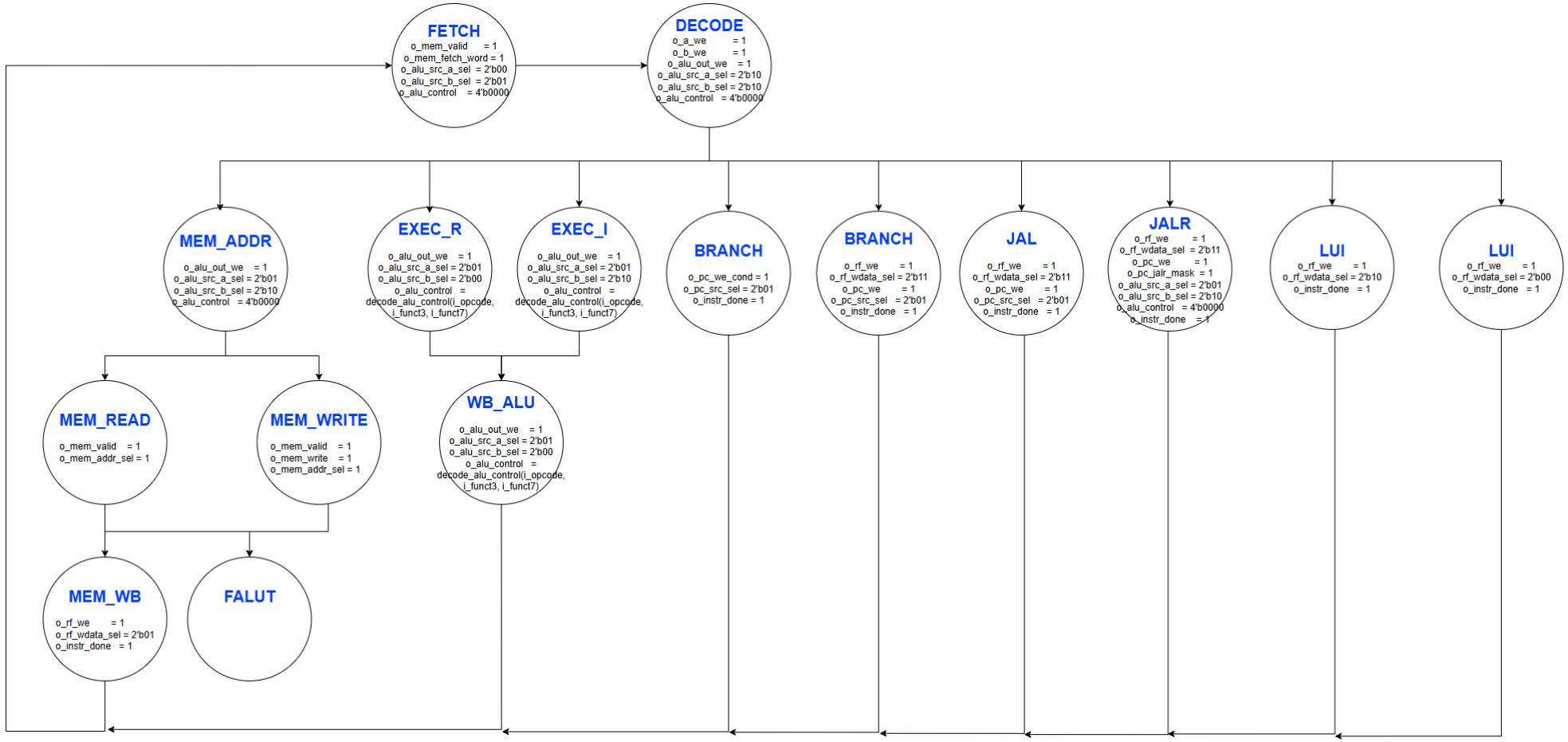
CPU

Block diagram



CPU

Multi Cycle - FSM



CPU

SPIKE 기반 기능 검증 자동화

RISC-V 자동화 기능 검증에서 널리 사용되는
golden reference ISA simulator

```
id@DESKTOP-7CFQ9ND:~$ spike -h
Spike RISC-V ISA Simulator 1.1.1-dev
```

Script 파일

- bootstrap.sh
- check_env.sh
- clean_outputs.sh
- clean_zone_files.sh
- finalize_handoff.sh
- run_all_isa.sh
- run_multicycle_spike_test.sh
- run_only_multicycle_test.sh
- run_smoke.sh
- run_system_test.sh
- run_test.sh
- sync_multicycle_cpu.sh

Manifest 파일

- isa.csv
- isa_rv32i.csv
- regression.csv
- regression_isa.csv
- regression_singlecycle_isa.csv
- workloads.csv

테스트 소스(ISA.s/ workload.c)를 Build

↓

SPIKE 실행 후, reference trace(commit log) 생성

↓

RTL 시뮬레이션 실행 후 RTL trace 생성

↓

두 trace file을 공통 format으로 정규화

↓

결과 비교

↓

모든 결과가 일치하고 테스트가 정상종료(Done)되면 PASS

결과 (raw) 파일

- compare_multicycle → isa → bisection
- compare_singlecycle
- only_multicycle

workloads → trapezoid →

이름	유형
_sim	파일 폴더
compare_result.csv	Microsoft Excel ...
compare_result.spike.csv	Microsoft Excel ...
hw_results.csv	Microsoft Excel ...
instr_trace.csv	Microsoft Excel ...
multicycle_workloads_trapezoid_compare_result.csv	Microsoft Excel ...
multicycle_workloads_trapezoid_compare_result.spike.csv	Microsoft Excel ...
multicycle_workloads_trapezoid_hw_results.csv	Microsoft Excel ...
multicycle_workloads_trapezoid_instr_trace.csv	Microsoft Excel ...
multicycle_workloads_trapezoid_program.bin	BIN 파일
multicycle_workloads_trapezoid_program.dump	DUMP 파일
multicycle_workloads_trapezoid_program.elf	ELF 파일
multicycle_workloads_trapezoid_program.mem	MEM 파일
multicycle_workloads_trapezoid_program_rtl.bin	BIN 파일
multicycle_workloads_trapezoid_program_rtl.dump	DUMP 파일
multicycle_workloads_trapezoid_program_rtl.elf	ELF 파일
multicycle_workloads_trapezoid_rtl_commit.log	텍스트 문서
multicycle_workloads_trapezoid_spike_trace.log	텍스트 문서
multicycle_workloads_trapezoid_xelab.log	텍스트 문서
multicycle_workloads_trapezoid_xsim.log	텍스트 문서
multicycle_workloads_trapezoid_xvlog.log	텍스트 문서
program.bin	BIN 파일
program.dump	DUMP 파일
program.elf	ELF 파일
program.mem	MEM 파일
program_rtl.bin	BIN 파일
program_rtl.dump	DUMP 파일
program_rtl.elf	ELF 파일

CPU

SPIKE

RV32I ISA 테스트 케이스 10종: 개별 어셈블리 테스트 프로그램

- 00_lui_auipc.S
- 01_jumps.S
- 02_branches.S
- 03_load_store.S
- 04_alu_imm.S
- 05_alu_reg.S
- 06_fence.S
- 06_jumps_edges.S
- 07_branch_matrix.S
- 08_load_store_edges.S
- 09_alu_edges.S

•lw
•lb, lbu
•lh, lhu
•sb, sh, sw
•sign extension / zero extension
•메모리 읽기/쓰기 경로

workload

- bisection.c
- trapezoid.c

bisection.c
- Root finding 알고리즘
- **반복적 분기/비교 기반 수치 연산**

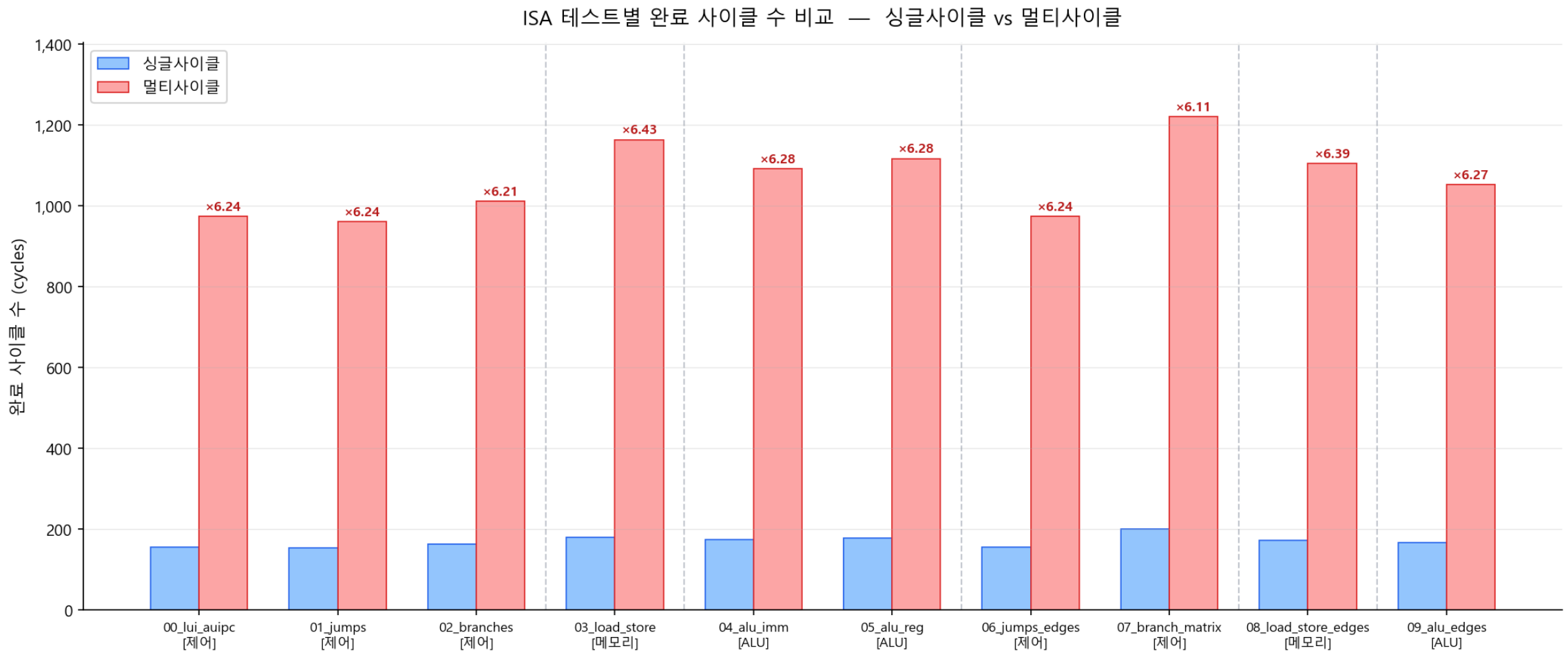
trapezoid.c
- 사다리꼴 적분
- **반복문 기반** 적분 계산
- **load/store와 산술 연산 포함**

```
[SMOKE] numerical workloads only :: singlecycle
=== BUILD ===
변환 완료: 265 words -> /mnt/c/rv32i_verify_runs/compare_singlecycle/workloads/trapezoid/program.mem
  address mode : rebased
  mem start    : @00000000
  _start @ 0x80000000
BUILD done
  ELF  : /mnt/c/rv32i_verify_runs/compare_singlecycle/workloads/trapezoid/program.elf
  MEM  : /mnt/c/rv32i_verify_runs/compare_singlecycle/workloads/trapezoid/program.mem
  DUMP : /mnt/c/rv32i_verify_runs/compare_singlecycle/workloads/trapezoid/program.dump
=== SPIKE ===
  spike log: /mnt/c/rv32i_verify_runs/compare_singlecycle/workloads/trapezoid/spike_trace.log
```

```
[rom_table] loading program.mem
DONE at cycle = 3110
[tb_program] results saved to hw_results.csv
$finish called at time : 31356 ns : File "C:/rv32i_verify_runs/compare_singlecycle/workloads/trapezoid/_sim/src/tb_progr
am.sv" Line 99
exit
INFO: [Common 17-206] Exiting xsim at Tue Mar 31 11:46:02 2026...
RTL done
  hw_results : /mnt/c/rv32i_verify_runs/compare_singlecycle/workloads/trapezoid/hw_results.csv
=== COMPARE ===
==================================================
 Spike Result Buffer vs RTL Result Buffer
 Total : 16
 PASS  : 16
 FAIL  : 0
 DONE  : YES
==================================================
 All compared result words match and DONE flag is present.
```

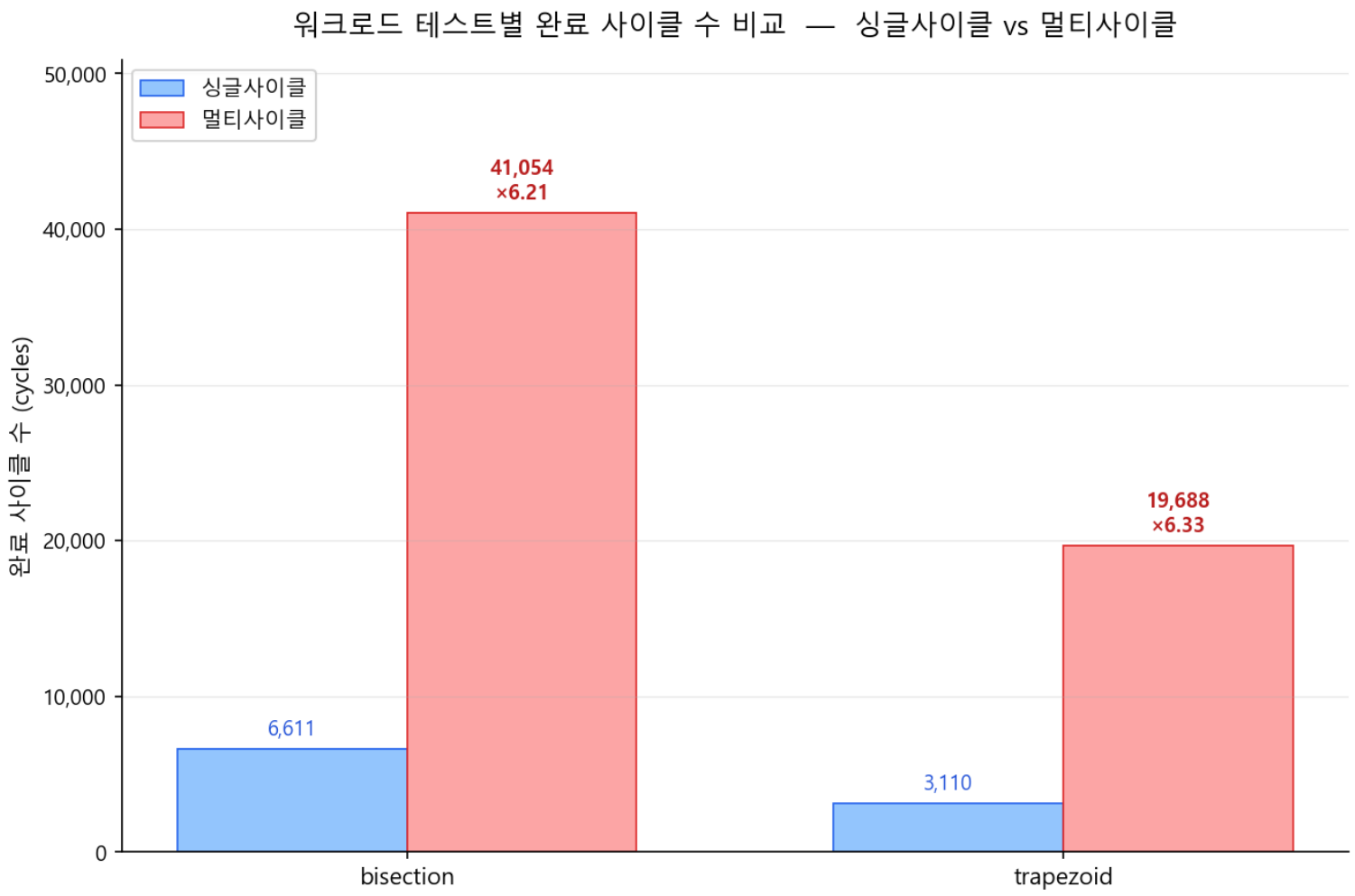
CPU

Single vs Multi



CPU

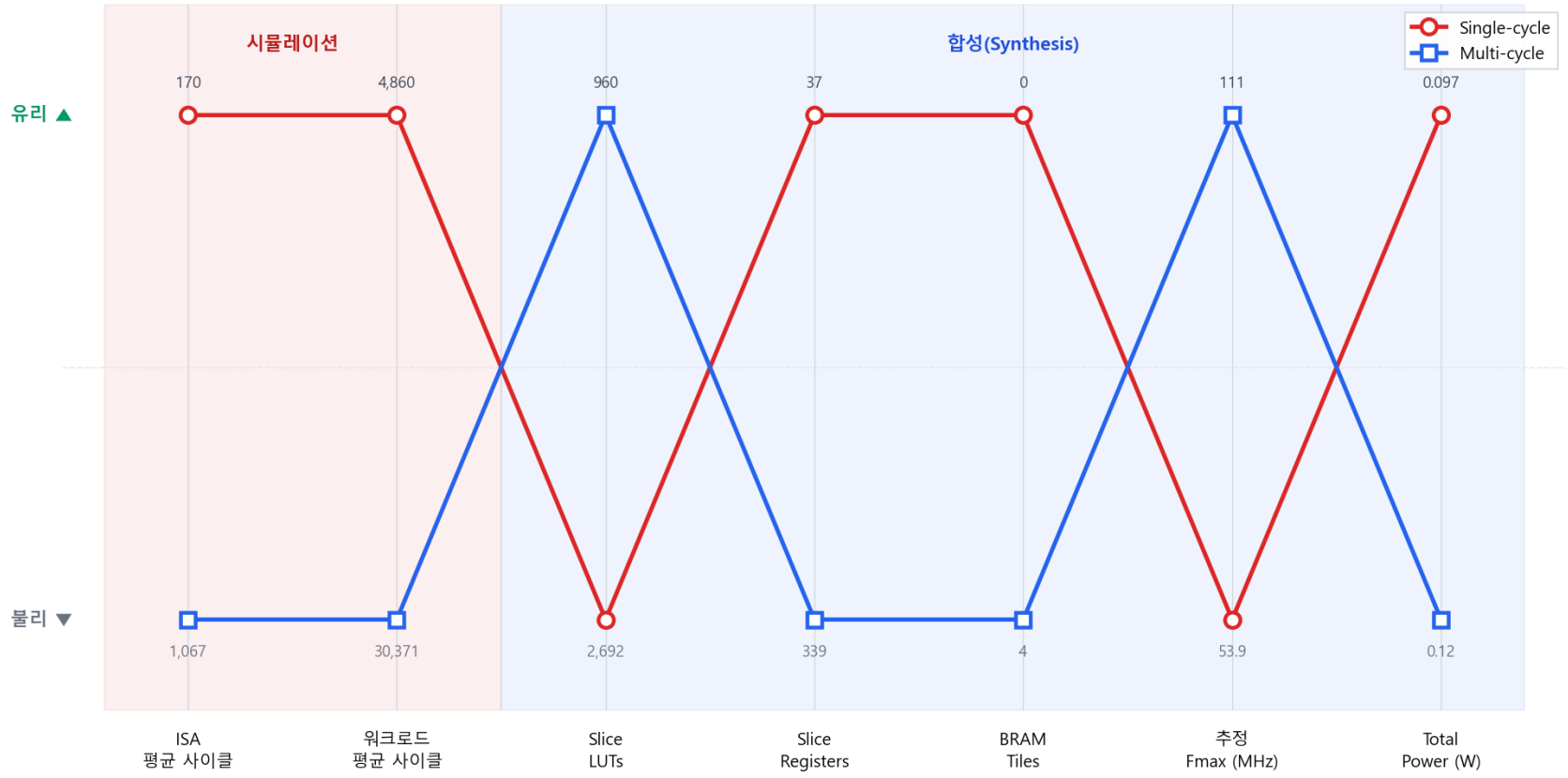
Single vs Multi



CPU

Single vs Multi

싱글사이클 vs 멀티사이클 — 독립 메트릭 7개 병렬 좌표 플롯 (PCP)



싱글 사이클 합성 자료

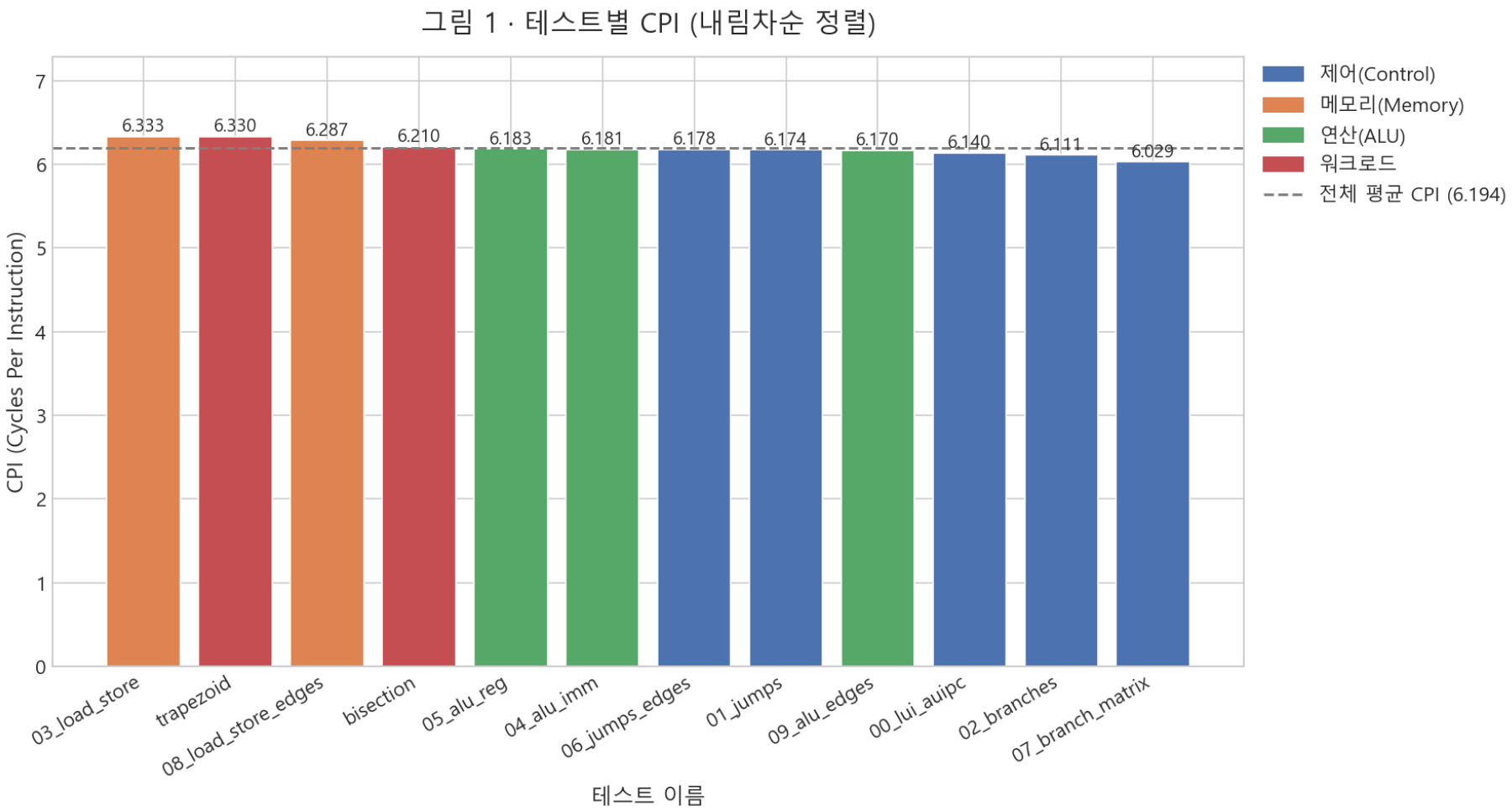
- Top_RV32I_Synth_power_routed.rpt
- Top_RV32I_Synth_timing_summary_routed.rpt
- Top_RV32I_Synth_utilization_routed.rpt

멀티 사이클 합성 자료

- Top_RV32I_Synth_power_routed.rpt
- Top_RV32I_Synth_timing_summary_routed.rpt
- Top_RV32I_Synth_utilization_routed.rpt

CPU

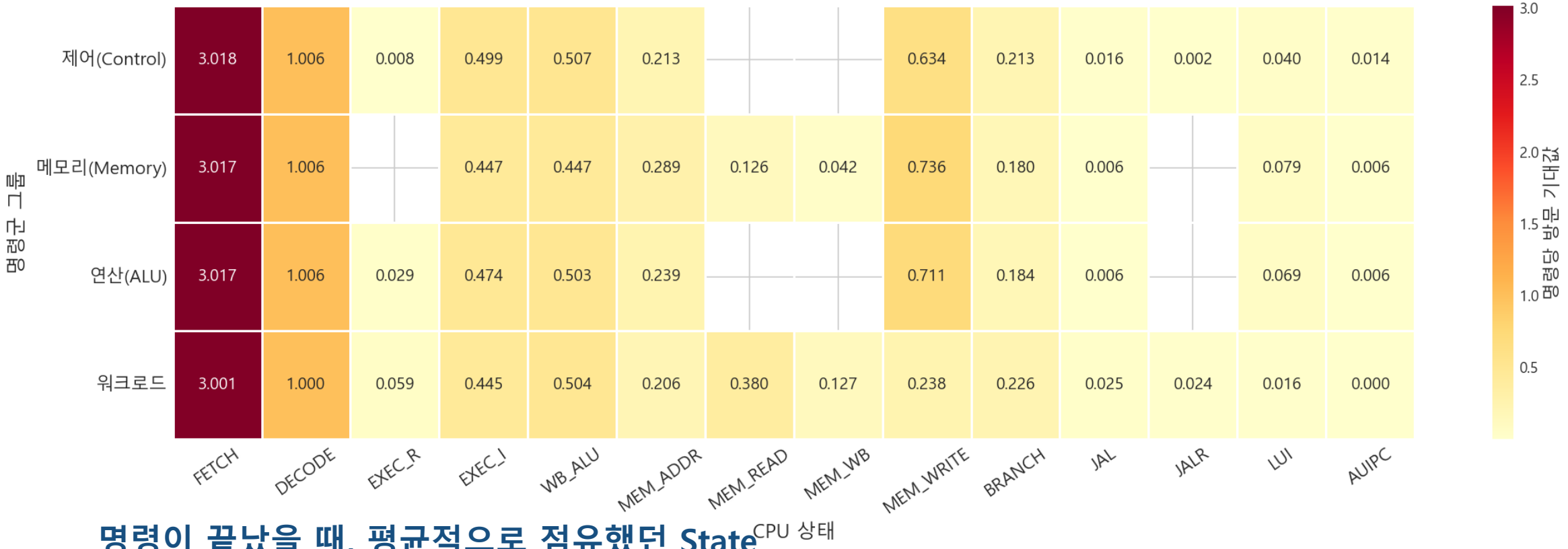
Multi-Cycle



CPU

Multi-Cycle

그림 4 · 그룹 x 상태 방문 기대값
(visits per completed instruction)

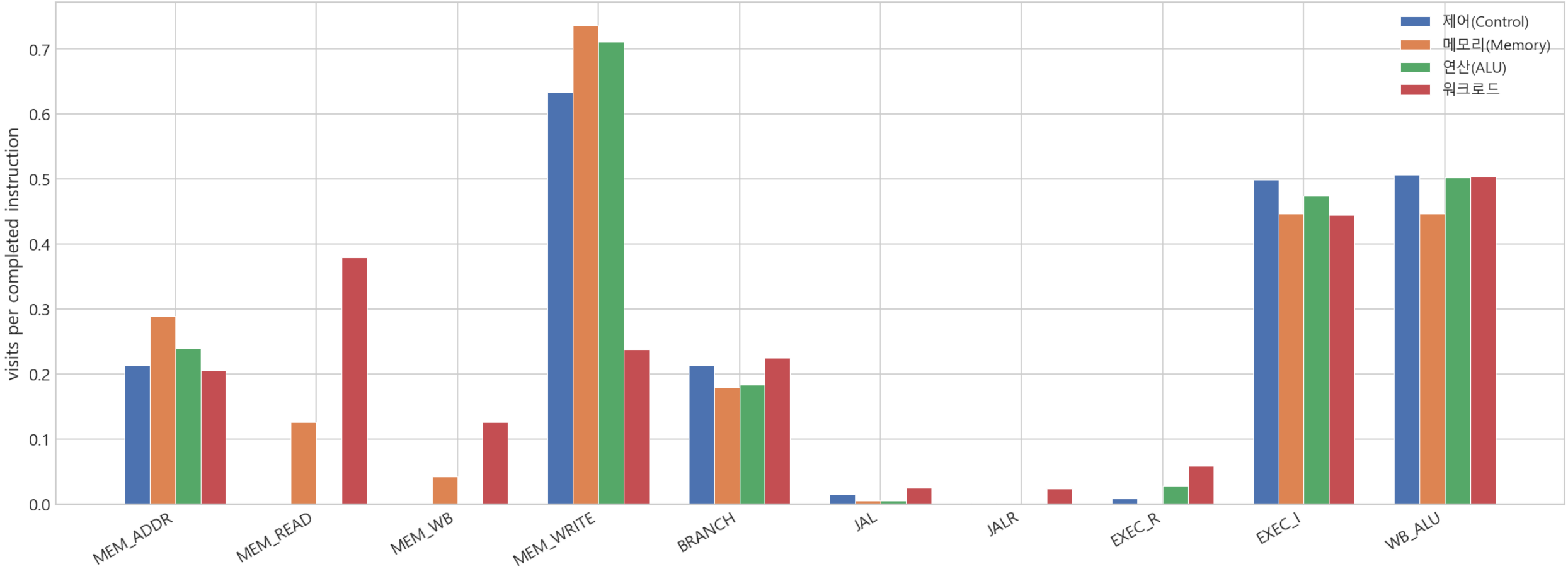


명령이 끝났을 때, 평균적으로 점유했던 State

CPU

Multi-Cycle

그림 5 · 주요 상태별 명령당 방문 기대값 — 그룹 비교



CPU

Multi-Cycle

그림 6 · 테스트 x 상태 점유율
(각 상태가 전체 사이클 중 차지하는 비율)

테스트 \ CPU 상태	FETCH	DECODE	EXEC_R	EXEC_I	WB_ALU	MEM_ADDR	MEM_READ	MEM_WB	MEM_WRITE	BRANCH	JAL	JALR	LUI	AUIPC
00_lui_auipc	0.486	0.162	0.002	0.075	0.077	0.038			0.113	0.033	0.001		0.009	0.004
01_jumps	0.486	0.162	0.002	0.076	0.078	0.037			0.111	0.033	0.002	0.001	0.007	0.003
02_branches	0.489	0.163		0.086	0.086	0.034			0.100	0.038	0.001		0.004	0.001
03_load_store	0.471	0.157		0.069	0.069	0.046	0.023	0.008	0.115	0.027	0.001		0.013	0.001
04_alu_imm	0.483	0.161		0.081	0.081	0.038			0.114	0.029	0.001		0.011	0.001
05_alu_reg	0.483	0.161	0.009	0.072	0.081	0.038			0.114	0.029	0.001		0.012	0.001
06_jumps_edges	0.486	0.162	0.002	0.078	0.080	0.037			0.110	0.033	0.002	0.001	0.006	0.003
07_branch_matrix	0.496	0.165	0.001	0.088	0.088	0.029			0.085	0.036	0.006		0.006	0.001
08_load_store_edges	0.475	0.158		0.071	0.071	0.044	0.016	0.005	0.116	0.029	0.001		0.012	0.001
09_alu_edges	0.484	0.161	0.005	0.076	0.081	0.038			0.113	0.030	0.001		0.010	0.001
bisection	0.483	0.161	0.010	0.073	0.083	0.031	0.057	0.019	0.036	0.037	0.004	0.004	0.002	0.000
trapezoid	0.474	0.158	0.008	0.068	0.076	0.037	0.068	0.023	0.043	0.034	0.004	0.004	0.003	0.000

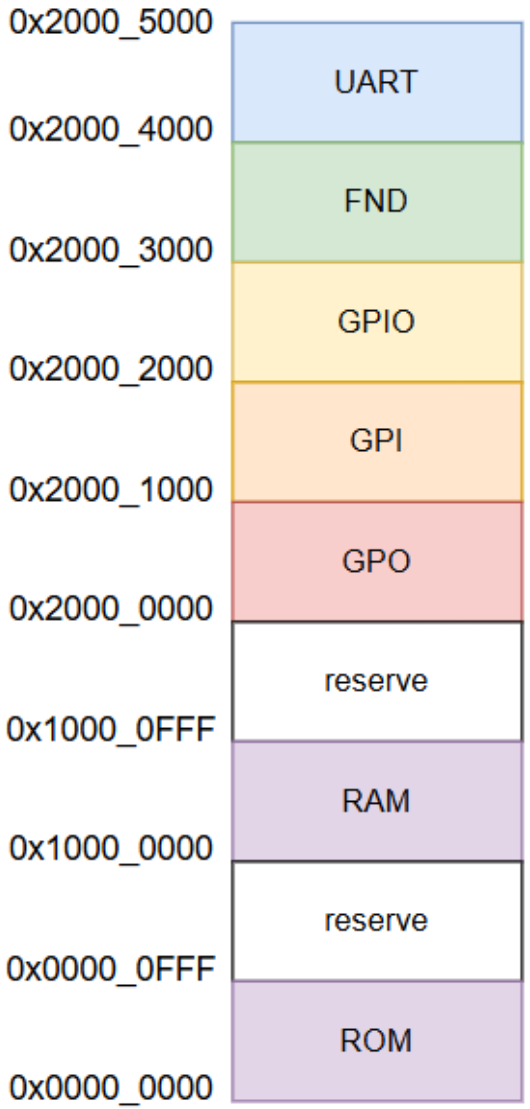
사이클 점유 비율: 0.1 – 0.2 – 0.3 – 0.4

AMBA Bus Protocol

AXI	AHB	APB
- 높은 성능을 요구하는 모듈간 통신에 적합 - 읽기, 쓰기 요청을 독립적으로 처리하여 동시성이 뛰어남	- 고속 데이터 전송 - CPU, Memory, DMA 같은 고성능 모듈 연결할 때 사용 - 파이프라인 구조를 지원	- 느린 주변 장치 (UART, 타이머)와 연결하는 데 사용 - 복잡도를 줄이고 전력을 절약하는데 중점

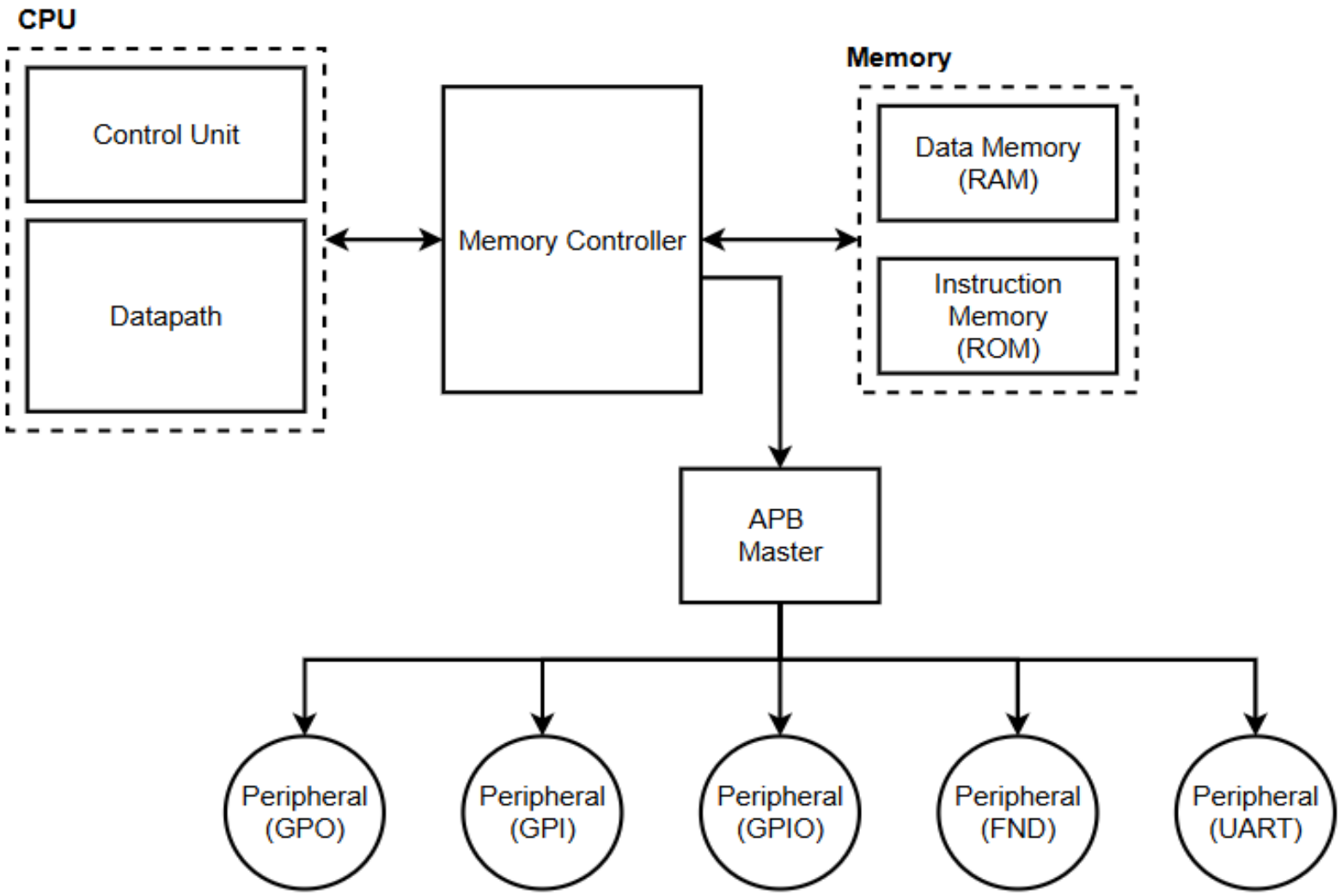
AMBA APB BUS 구현

Memory Mapping

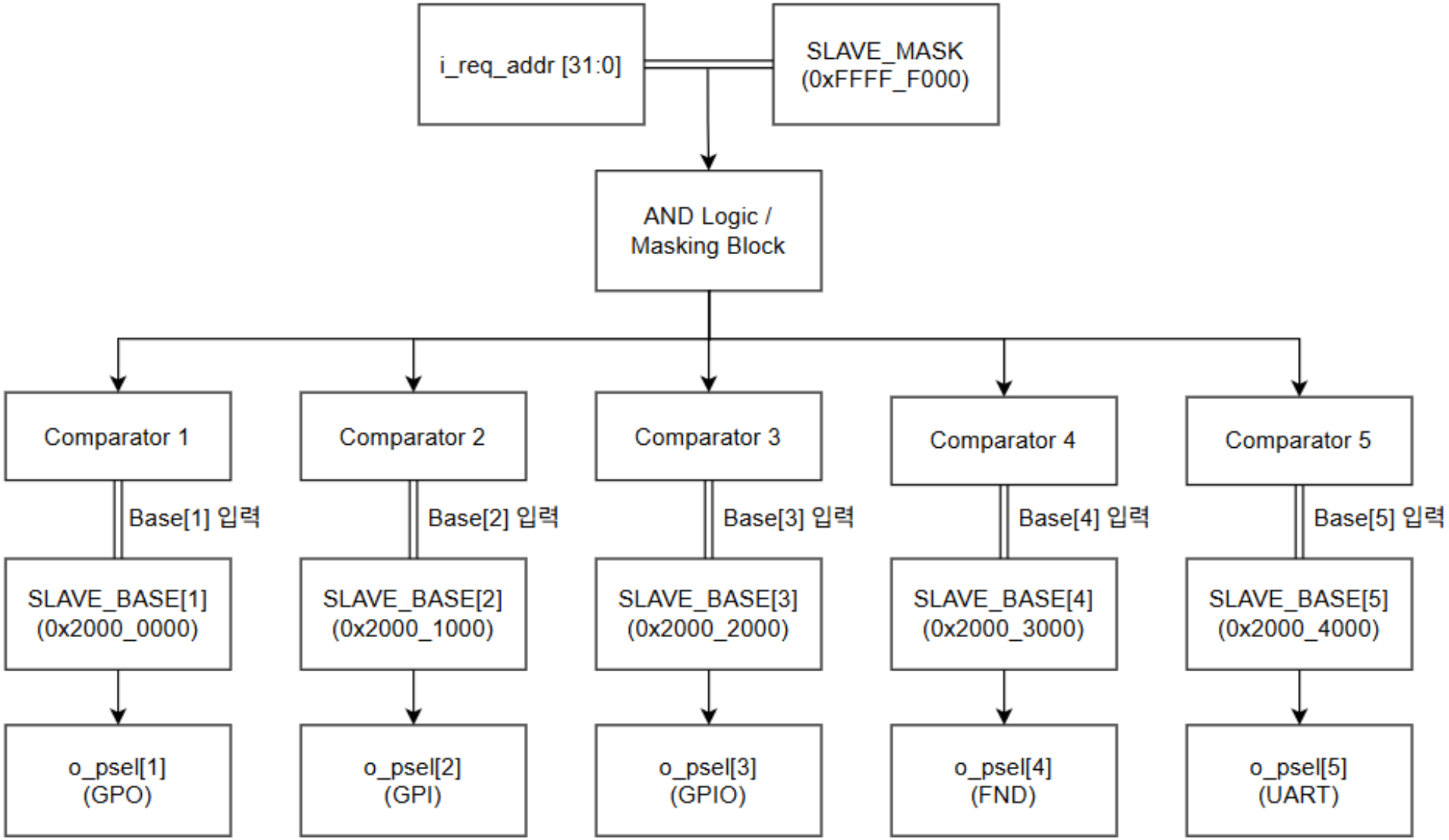


Peripheral	Base	Purpose
GPO	`0x2000_0000`	16-bit general-purpose output register block
GPI	`0x2000_1000`	16-bit general-purpose input register block
GPIO	`0x2000_2000`	16-bit bidirectional GPIO register block
FND	`0x2000_3000`	4-digit seven-segment counter peripheral with simple run/stop control
UART	`0x2000_4000`	UART TX/RX peripheral with status and error clear bits

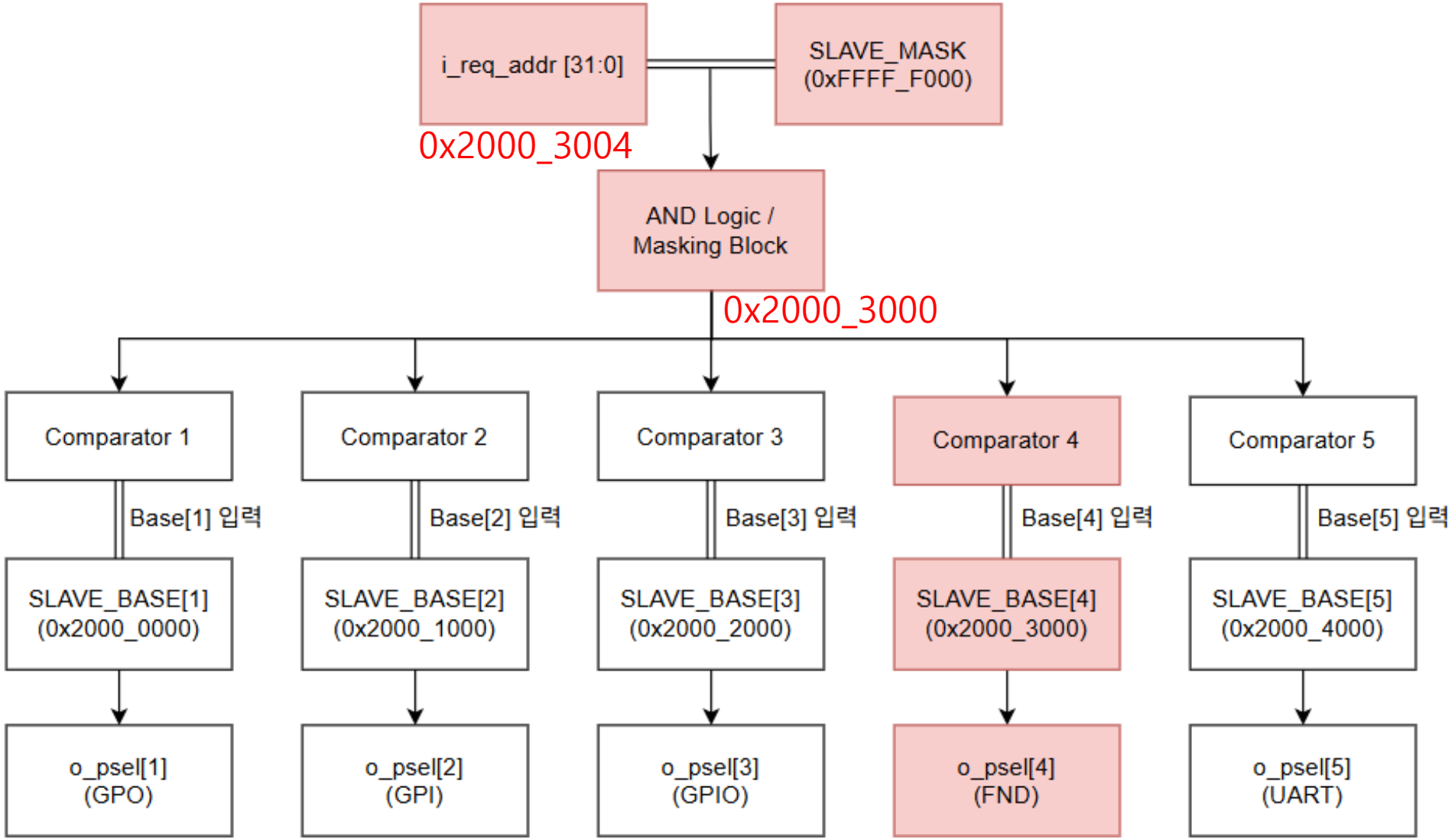
Block Diagram



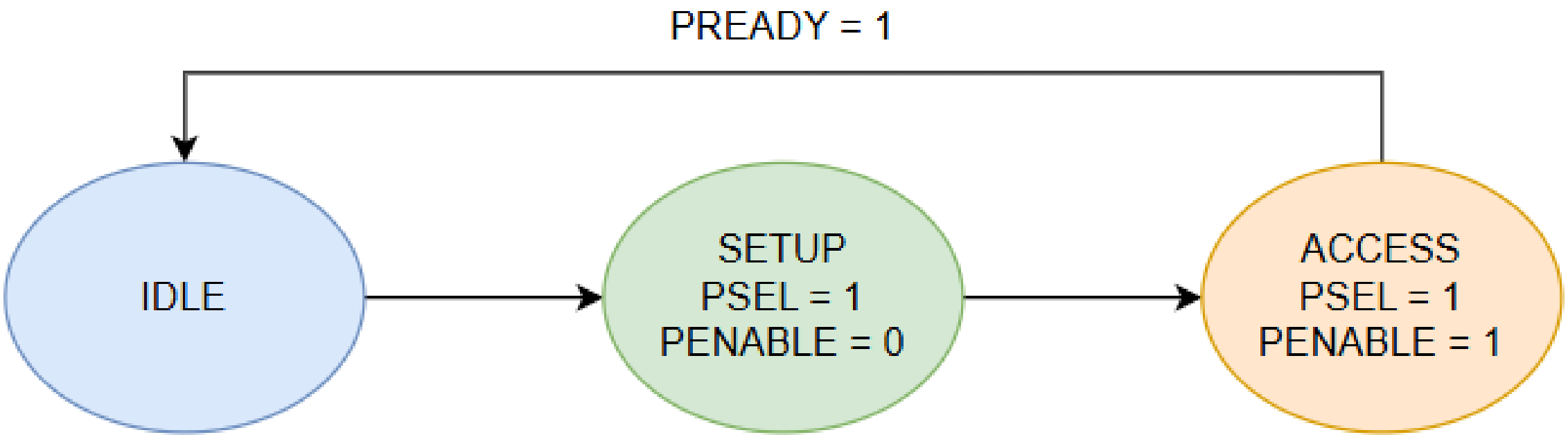
APB Bridge Address Decoding



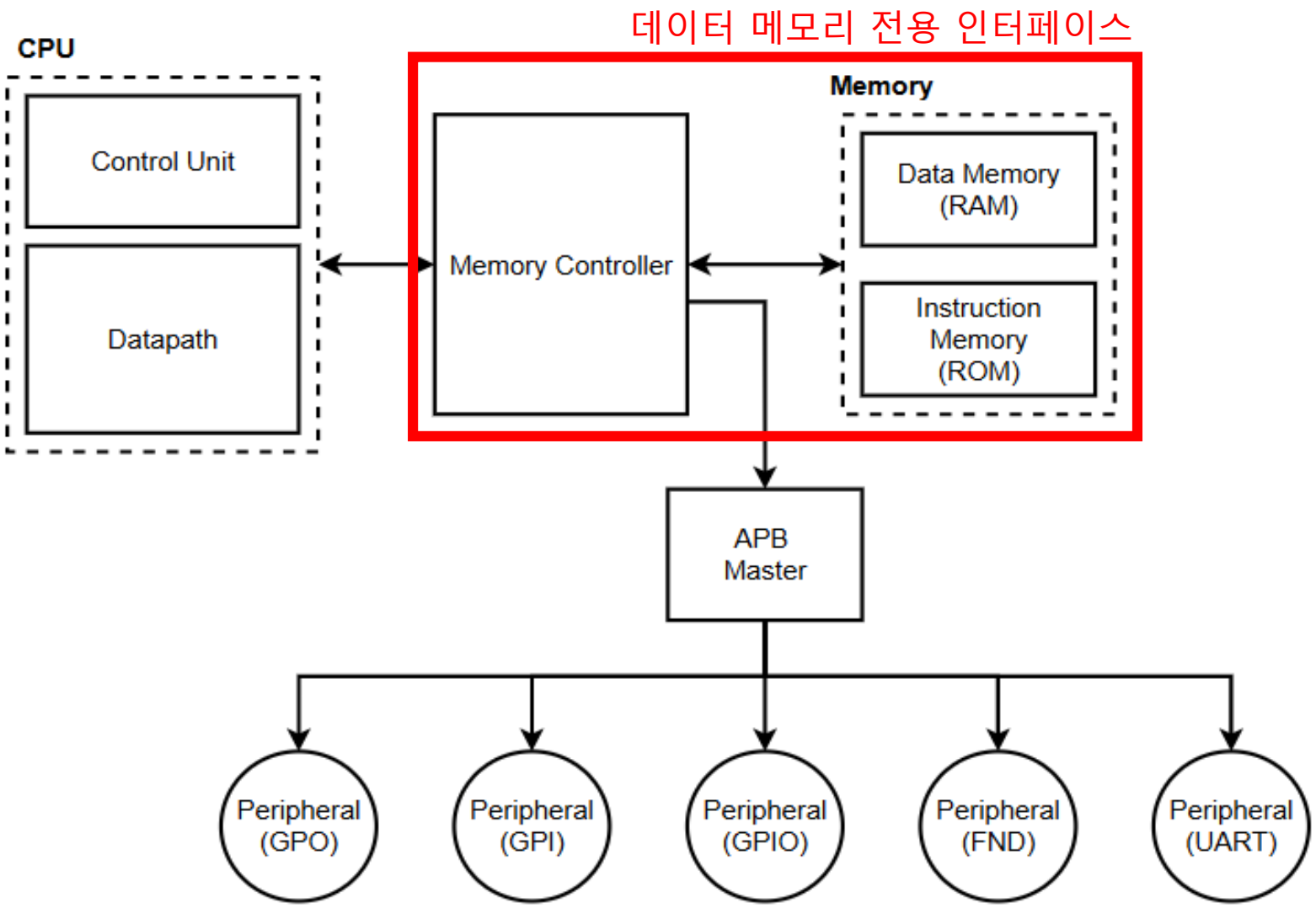
APB Bridge Address Decoding



APB Master FSM



Simulation Scenario



Simulation Scenario

1. [전체 흐름] 통합 시스템 기본 동작 검증

동일한 메모리 주소(0x1000_0004)에 대한 연속적인 쓰기(Store) 및 읽기(Load) 트랜잭션이 데드락(멈춤) 없이 수행되는지 확인

2. [내부 동작] 데이터 쓰기(Store) 및 라우팅 검증

컨트롤러의 주소 해독을 거쳐, 실제 RAM 내부 배열(Array)의 정확한 인덱스에 데이터가 물리적으로 기록되는지 검증

3. [타이밍] 메모리 읽기(Load) 및 1-Cycle 지연 증명

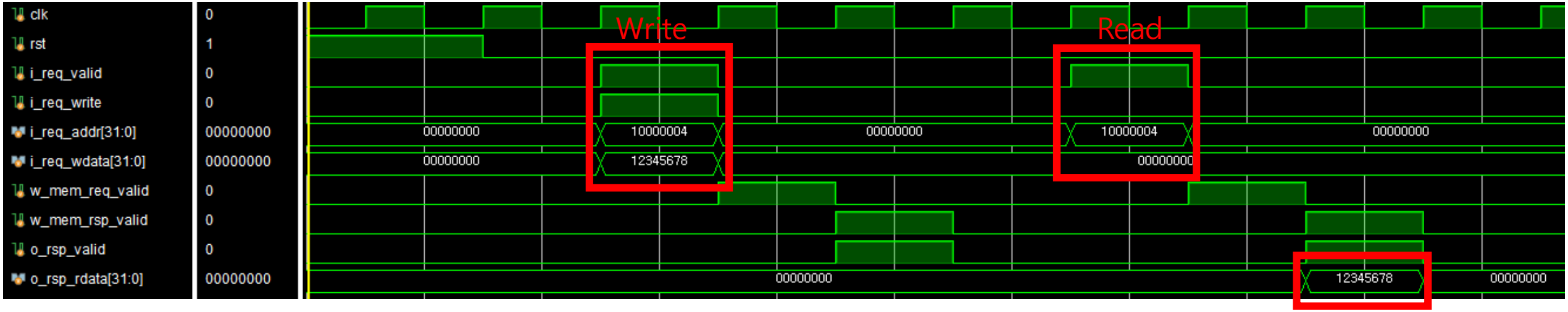
동기식 RAM의 1클럭 읽기 지연(Read Latency) 특성에 맞춰, 컨트롤러가 정확한 타이밍에 데이터를 샘플링하는지 확인

4. [설계 견고함] Read/Write 프로토콜 명확화

쓰기 응답(Ack)과 읽기 데이터(Read Data) 응답을 완벽히 분리하여, 버스 상의 더미 데이터 출력을 원천 차단했음을 증명

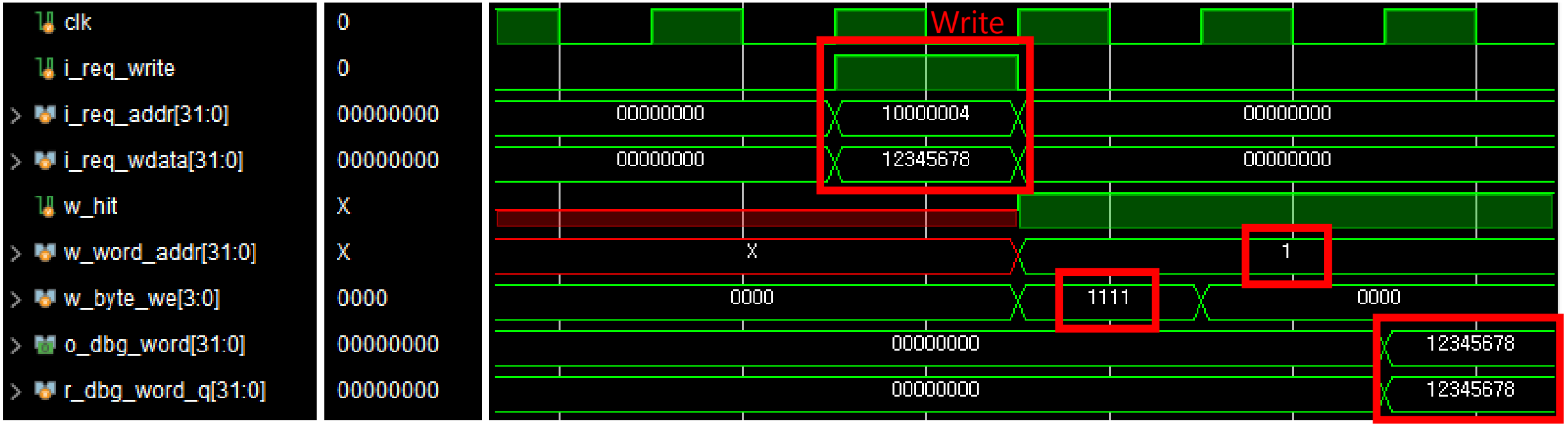
Simulation

통합 시스템 메모리 검증



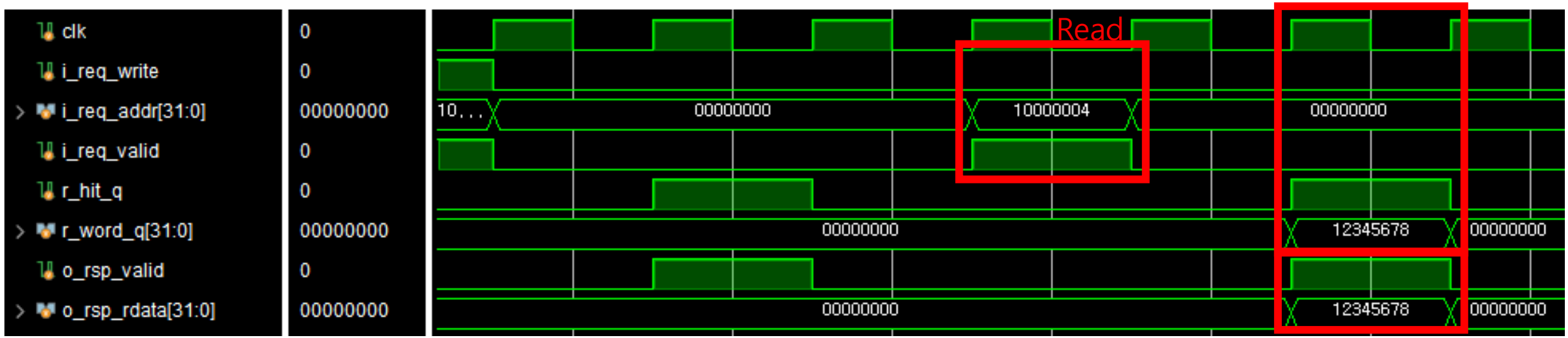
Simulation

데이터 쓰기 동작 및 내부 메모리 라우팅 및 디코딩 검증



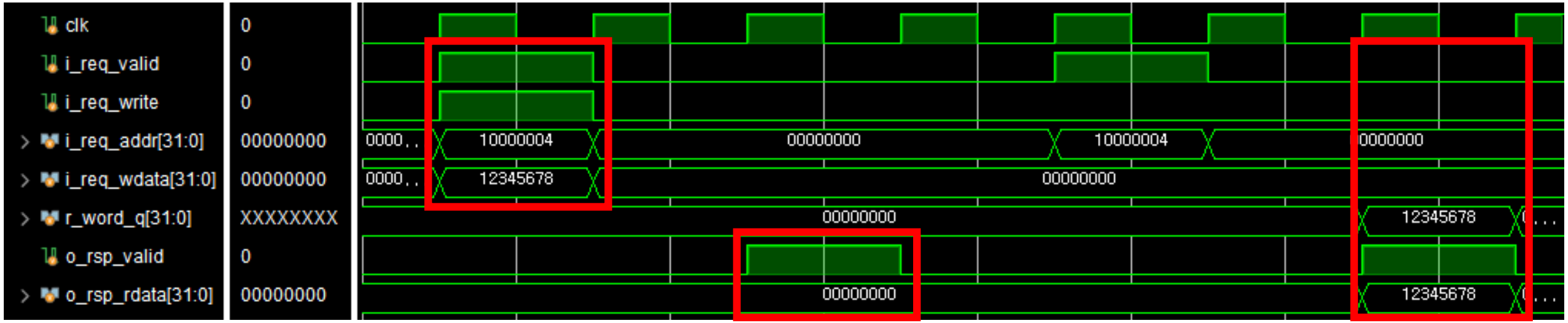
Simulation

메모리 읽기 동작 및 1-Cycle 지연 검증



Simulation

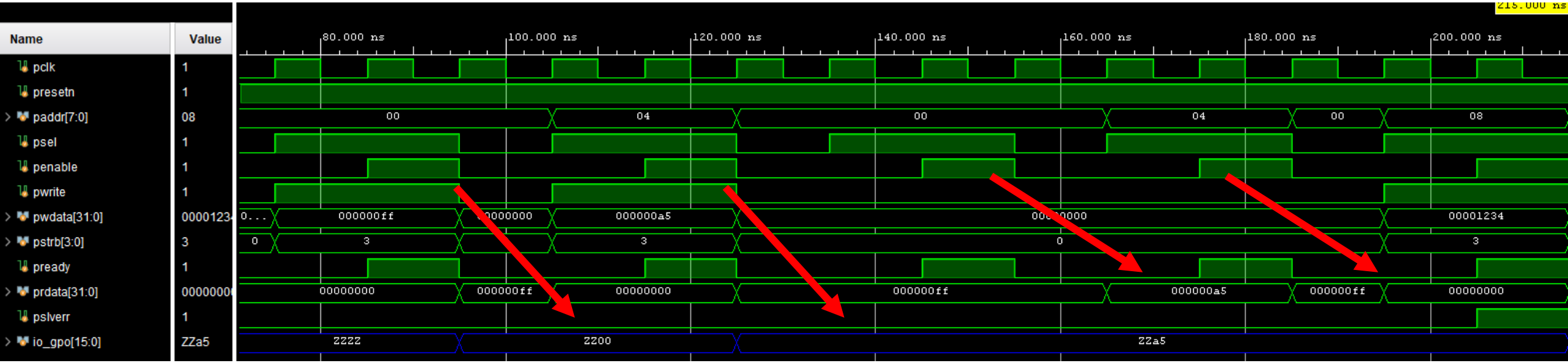
Read/Write 프로토콜의 명확한 분리



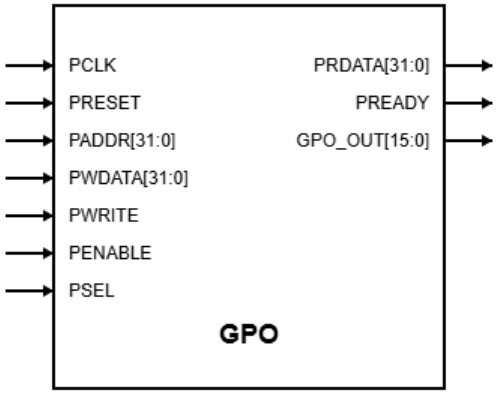
Register Map

T	NAME	Offset	Range	Field	Reset	Access	Description
S	GPO						General-purpose output register page
R	GPO_CTL	0x2000_0000	[15:0]		0x0000	R/W	Per-bit output-enable mask. 1 drives the output bit, 0 leaves the bit high-Z.
F	GPO_CTL	0x2000_0000	[15:0]	GPO_CTL[15:0]	0x0000	R/W	Output-enable mask for 16 GPO bits.
R	GPO_ODATA	0x2000_0004	[15:0]		0x0000	R/W	Output data written to the bits enabled by GPO_CTL.
F	GPO_ODATA	0x2000_0004	[15:0]	GPO_ODATA[15:0]	0x0000	R/W	Driven value for each enabled GPO output.
S	GPI						General-purpose input register page
R	GPI_CTL	0x2000_1000	[15:0]		0x0000	R/W	Input-enable mask for the sampled switch/input bus.
F	GPI_CTL	0x2000_1000	[10:0]	GPI_CTL[10:0]	0x0000	R/W	Enable mask for active switch inputs in the current top-level build.
R	GPI_IDATA	0x2000_1004	[15:0]		0x0000	R	Masked input sample returned to software.
F	GPI_IDATA	0x2000_1004	[3:0]	UART_APB_SEL	0x0	R	Requested UART baud code from external switches.
F	GPI_IDATA	0x2000_1004	[4]	UART_SOURCE_SEL	0x0	R	0 = use switch baud source, 1 = use APB-programmed baud source.
F	GPI_IDATA	0x2000_1004	[5]	FND_RUN_REQ	0x0	R	External request bit mirrored to the seven-segment run control flow.
F	GPI_IDATA	0x2000_1004	[9:6]	GPIO_PATTERN	0x0	R	External GPIO pattern input used by the demo firmware.
F	GPI_IDATA	0x2000_1004	[10]	USER_BIT	0x0	R	Extra user-controlled switch bit. Not consumed by the current demo flow.

GPO

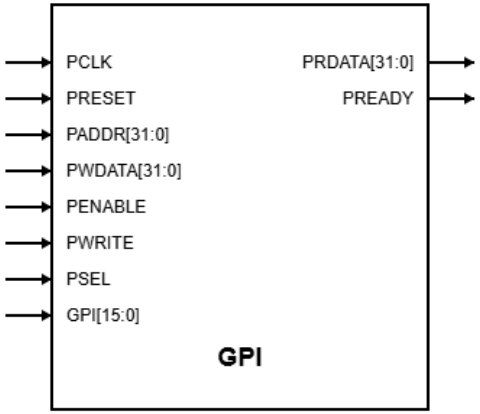



```
[65000][GPO_TB] READ  addr=00 data=00000000 slverr=0 io_gpo=zzzz
[95000][GPO_TB] WRITE addr=00 data=000000ff slverr=0 io_gpo=zzzz
[125000][GPO_TB] WRITE addr=04 data=000000a5 slverr=0 io_gpo=zz00
[155000][GPO_TB] READ  addr=00 data=000000ff slverr=0 io_gpo=zza5
[185000][GPO_TB] READ  addr=04 data=000000a5 slverr=0 io_gpo=zza5
[215000][GPO_TB] WRITE addr=08 data=00001234 slverr=1 io_gpo=zza5
```



GPI

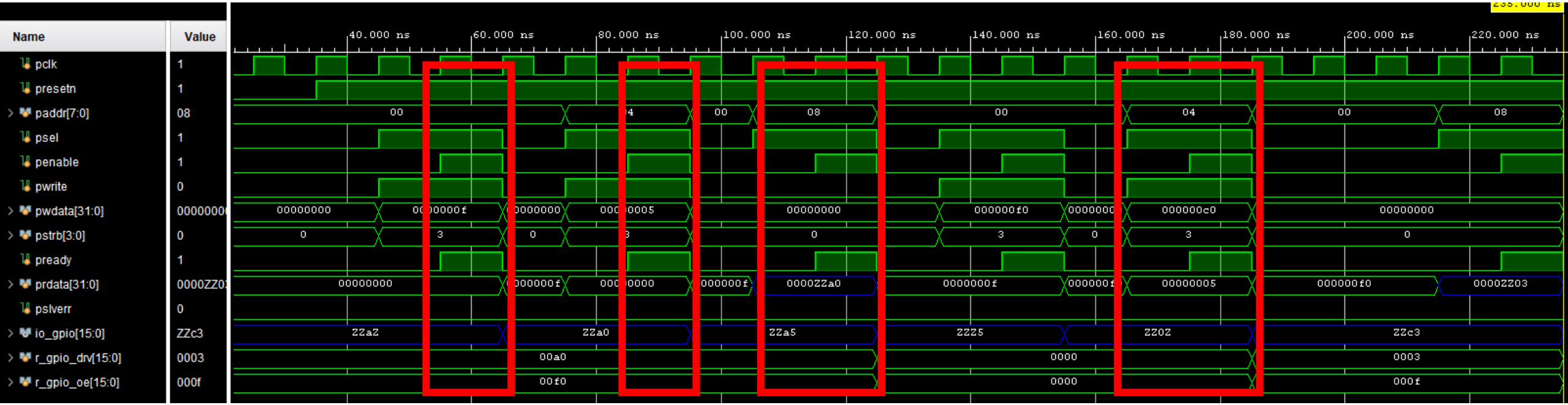
```
[65000][GPI_TB] READ  addr=00 data=00000000 slverr=0 i_gpi=a55a
[95000][GPI_TB] WRITE addr=00 data=000000ff slverr=0 i_gpi=a55a
[125000][GPI_TB] READ  addr=04 data=0000005a slverr=0 i_gpi=a55a
[175000][GPI_TB] READ  addr=04 data=000000a5 slverr=0 i_gpi=00a5
[205000][GPI_TB] WRITE addr=04 data=0000ffff slverr=1 i_gpi=00a5
```



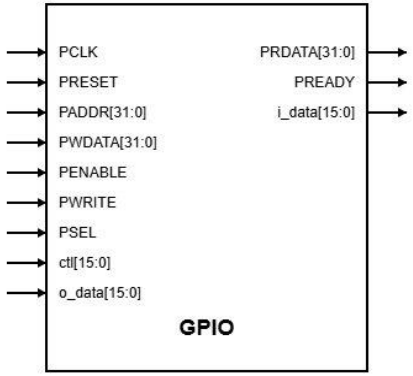
Register Map

T	NAME	Offset	Range	Field	Reset	Access	Description
S	**GPIO**						**Bidirectional GPIO register page**
R	GPIO_CTL	0x2000_2000	[15:0]		0x0000	R/W	Direction control. 1 = output, 0 = input.
F	GPIO_CTL	0x2000_2000	[3:0]	GPIO_CTL[3:0]	0x0	R/W	Active GPIO direction bits connected to the top-level IO bus.
F	GPIO_CTL	0x2000_2000	[15:4]	Reserved	0x000	R/W	Present in the 16-bit register image but not routed to top-level IO in this build.
R	GPIO_ODATA	0x2000_2004	[15:0]		0x0000	R/W	Output value for GPIO bits configured as outputs.
F	GPIO_ODATA	0x2000_2004	[3:0]	GPIO_ODATA[3:0]	0x0	R/W	Driven output data for the active GPIO pins.
R	GPIO_IDATA	0x2000_2008	[15:0]		0x0000	R	Observed input value for the GPIO bus.
F	GPIO_IDATA	0x2000_2008	[3:0]	GPIO_IDATA[3:0]	0x0	R	Input sample from the active GPIO pins.
S	**FND**						**Seven-segment counter control page**
R	FND_RUN	0x2000_3000	[0]		0x0	R/W	Run/stop bit for the four-digit seven-segment counter.
F	FND_RUN	0x2000_3000	[0]	FND_RUN_EN	0x0	R/W	0 = stop counter, 1 = run counter.

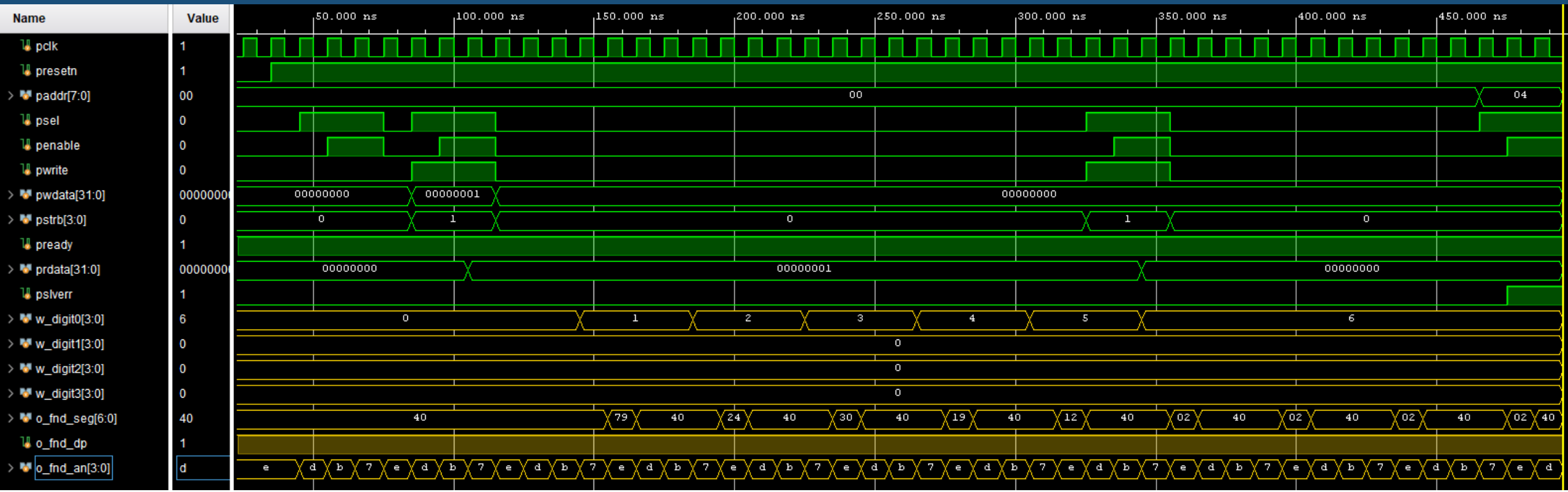
GPIO



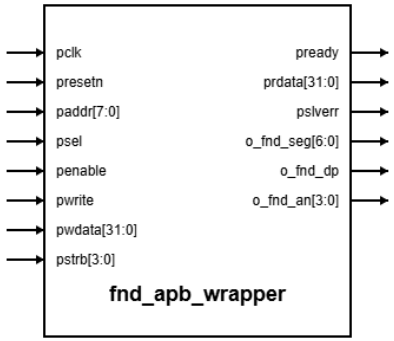

```
[65000][GPIO_TB] WRITE addr=00 data=0000000f slverr=0 io_gpio=zzaz
[95000][GPIO_TB] WRITE addr=04 data=00000005 slverr=0 io_gpio=zza0
[125000][GPIO_TB] READ  addr=08 data=0000zza0 slverr=0 io_gpio=zza5
[155000][GPIO_TB] WRITE addr=00 data=000000f0 slverr=0 io_gpio=zzz5
[185000][GPIO_TB] WRITE addr=04 data=000000c0 slverr=0 io_gpio=zz0z
[235000][GPIO_TB] READ  addr=08 data=0000zz03 slverr=0 io_gpio=zzc3
```



FND




```
[75000][FND_TB] READ  addr=00 data=00000000 slverr=0 run=0 count=0 an=0111 seg=1000000
[115000][FND_TB] WRITE addr=00 data=00000001 slverr=0 run=1 count=0 an=0111 seg=1000000
[355000][FND_TB] WRITE addr=00 data=00000000 slverr=0 run=0 count=6 an=0111 seg=1000000
[495000][FND_TB] READ  addr=04 data=00000000 slverr=1 run=0 count=6 an=1101 seg=1000000
```



Register Map

4	T	NAME	Offset	Range	Field	Reset	Access	Description
30	S	UART						UART TX/RX register page
31	R	UART_ID	0x2000_4000	[31:0]		0x5541_5254	R	Fixed peripheral identifier. ASCII value corresponds to 'UART'.
32	F	UART_ID	0x2000_4000	[31:0]	UART_ID	0x5541_5254	R	Constant identification value for software probe/readback.
33	R	UART_STATUS	0x2000_4004	[6:0]		0x0000_000A	R	FIFO state, TX busy, and sticky RX error flags.
34	F	UART_STATUS	0x2000_4004	[0]	TX_FULL	0	R	TX FIFO full indicator.
35	F	UART_STATUS	0x2000_4004	[1]	TX_EMPTY	1	R	TX FIFO empty indicator.
36	F	UART_STATUS	0x2000_4004	[2]	RX_FULL	0	R	RX FIFO full indicator.
37	F	UART_STATUS	0x2000_4004	[3]	RX_EMPTY	1	R	RX FIFO empty indicator.
38	F	UART_STATUS	0x2000_4004	[4]	TX_BUSY	0	R	UART transmitter is sending an active frame.
39	F	UART_STATUS	0x2000_4004	[5]	RX_OVERFLOW	0	R	Sticky flag set when a new RX byte arrives while RX FIFO is full.
40	F	UART_STATUS	0x2000_4004	[6]	FRAME_ERROR	0	R	Sticky flag set when the received stop bit check fails.
41	R	UART_TXDATA	0x2000_4008	[7:0]		0x00	W	Write one byte into the TX FIFO when space is available.
42	F	UART_TXDATA	0x2000_4008	[7:0]	TXDATA	0x00	W	Transmit byte payload. Write while FIFO full raises PSLVERR.
43	R	UART_RXDATA	0x2000_400C	[7:0]		0x00	R	Read and pop one byte from the RX FIFO.
44	F	UART_RXDATA	0x2000_400C	[7:0]	RXDATA	0x00	R	Received byte payload. Read while FIFO empty raises PSLVERR.
45	R	UART_CONTROL	0x2000_4010	[1:0]		0x0000_0000	R/W	Sticky error clear register with status mirror behavior.
46	F	UART_CONTROL	0x2000_4010	[0]	CLR_OVERFLOW	0	R/W	Write 1 to clear the RX overflow sticky flag.
47	F	UART_CONTROL	0x2000_4010	[1]	CLR_FRAME	0	R/W	Write 1 to clear the frame-error sticky flag.
48	R	UART_BAUDCFG	0x2000_4014	[11:0]		0x0000_0005 *	R/W	Selects APB-vs-switch baud source and exposes the active baud code.
49	F	UART_BAUDCFG	0x2000_4014	[3:0]	APB_SEL	0x5	R/W	APB-programmed baud selection code.
50	F	UART_BAUDCFG	0x2000_4014	[4]	SOURCE_SEL	0	R/W	0 = use external switch baud select, 1 = use APB baud select.
51	F	UART_BAUDCFG	0x2000_4014	[11:8]	ACTIVE_SEL	switch	R	Read-only active baud select code currently used by the UART core.

UART 누적 방식 분주

$$K = \text{round}\left(\frac{f_{\text{tick,target}} \cdot 2^W}{f_{\text{clk}}}\right)$$

- K: `phase_inc`
- W: `ACC_WIDTH = 24`
- f_{clk}: 시스템 클럭 (`100 MHz`)

$$\phi[n+1] = (\phi[n] + K) \bmod 2^W$$

$$\text{tick}[n] = \left\lfloor \frac{\phi[n] + K}{2^W} \right\rfloor$$

n: clk index
tick[n] : 0 or 1

```
function automatic [ACC_WIDTH-1:0] calc_phase_inc;
        input integer baud_rate;
        reg   [63:0] target_tick_hz;
        reg   [63:0] phase_inc_full;
        begin
            target_tick_hz = baud_rate * OVERSAMPLE;
            phase_inc_full = ((target_tick_hz << ACC_WIDTH) + (SYS_CLK / 2)) / SYS_CLK;
            if (phase_inc_full == 0) begin
                calc_phase_inc = {{(ACC_WIDTH-1){1'b0}}, 1'b1};
            end else begin
                calc_phase_inc = phase_inc_full[ACC_WIDTH-1:0];
            end
        end
    endfunction
assign phase_sum = {1'b0, phase_acc} + {1'b0, phase_inc};
```

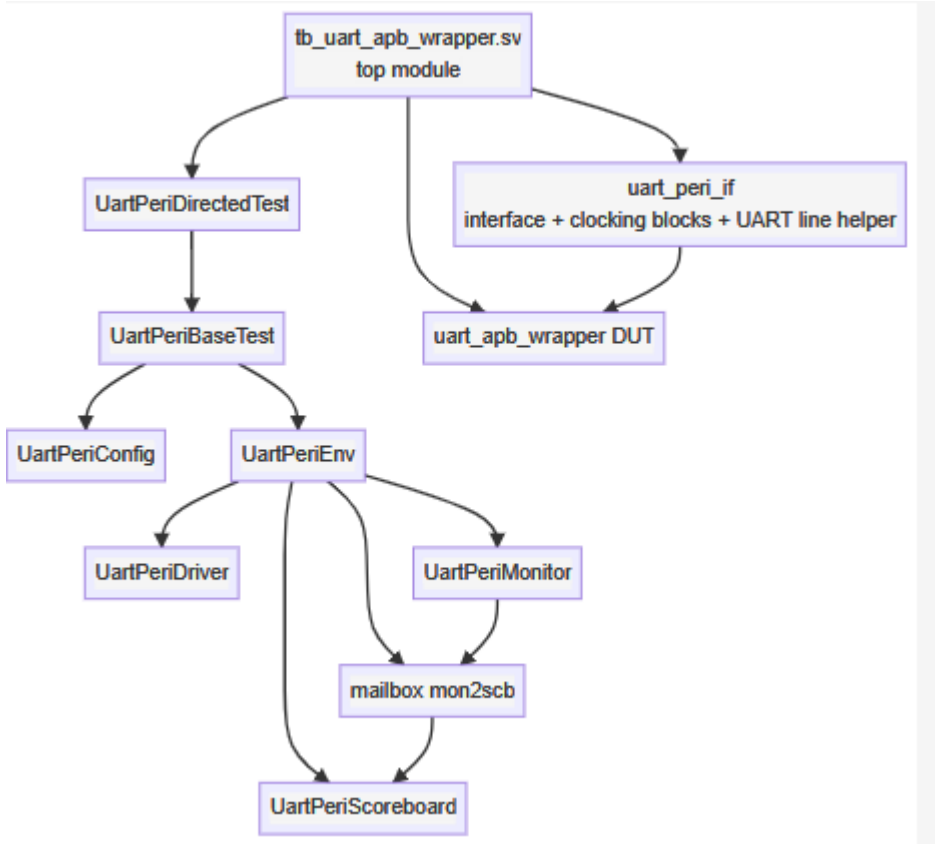
$$\frac{100,000,000}{1,843,200} \approx 54.253472 \text{ clocks}$$

UART 누적 방식 오차

기존 방식과 변경 방식 오차 비교

목표 baud	기존 실제 baud	기존 오차 [%]	변경 실제 baud	변경 오차 [%]
9,600	9,600.6144	+0.006400	9,600.0731	+0.000761
115,200	115,740.7407	+0.469393	115,200.1321	+0.000115
230,400	231,481.4815	+0.469393	230,399.8917	-0.000047
460,800	480,769.2308	+4.333600	460,800.1560	+0.000034
921,600	1,041,666.6667	+13.028067	921,599.9395	-0.000007
1,000,000	1,041,666.6667	+4.166667	1,000,000.1639	+0.000016
2,000,000	2,083,333.3333	+4.166667	1,999,999.9553	-0.000002
3,000,000	3,125,000.0000	+4.166667	3,000,000.1192	+0.000004

Simulation

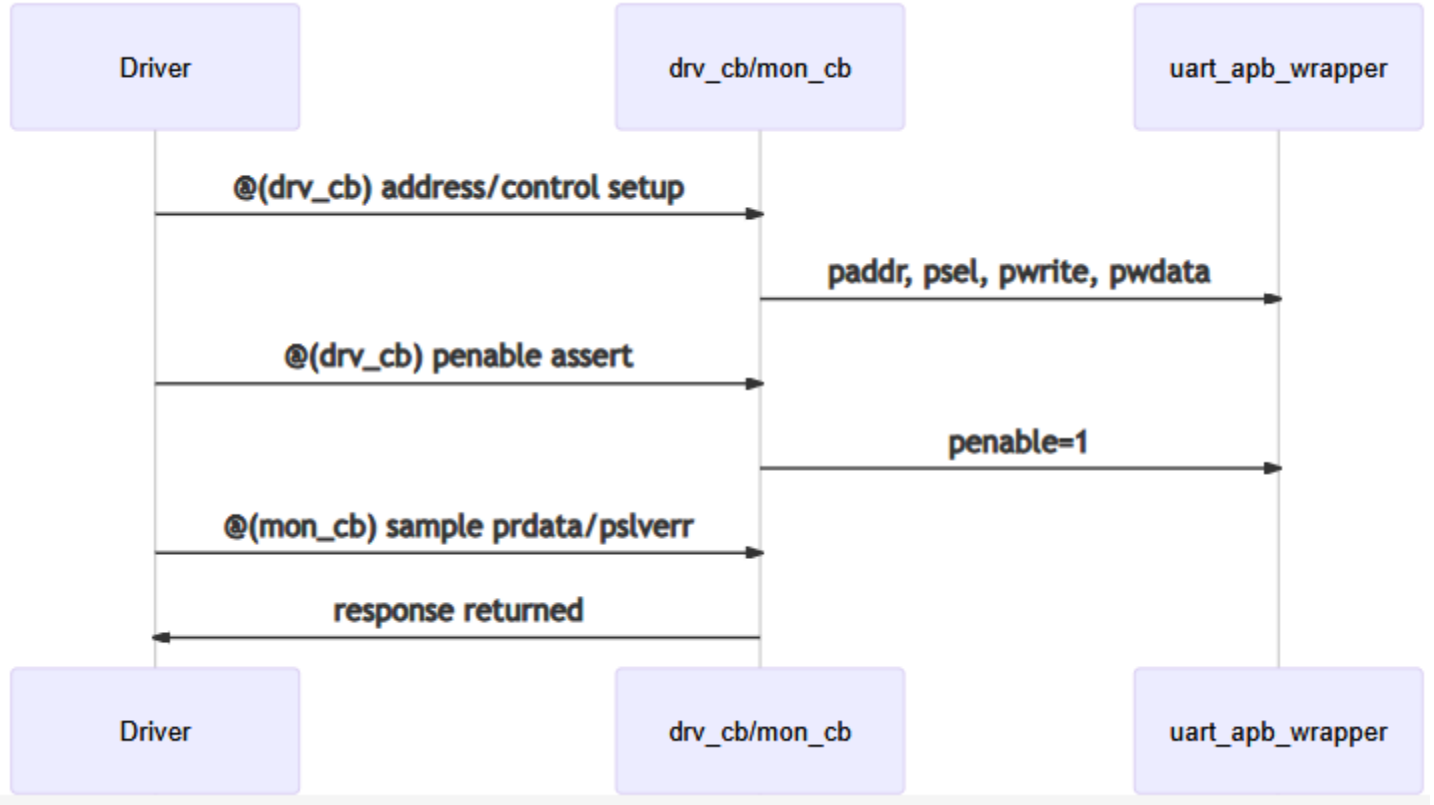


파일 / 클래스	역할
tb_uart_apb_wrapper	clock/reset 생성, DUT instantiation, test 실행, assertion 보유
uart_peri_if	APB signal 묶음, clocking block, UART RX/TX helper task
UartPeriConfig	baud, bit period, jitter, timeout 설정
UartPeriDriver	APB read/write, RX line 자극 주입
UartPeriMonitor	DUT의 o_uart_tx를 실제 serial line에서 복원
UartPeriScoreboard	expected TX와 monitor가 잡은 actual TX 비교
UartPeriEnv	driver/monitor/scoreboard를 묶어 병렬 실행
UartPeriBaseTest	공통 check 함수, env 생성, 공통 run flow
UartPeriDirectedTest	실제 directed scenario 구현

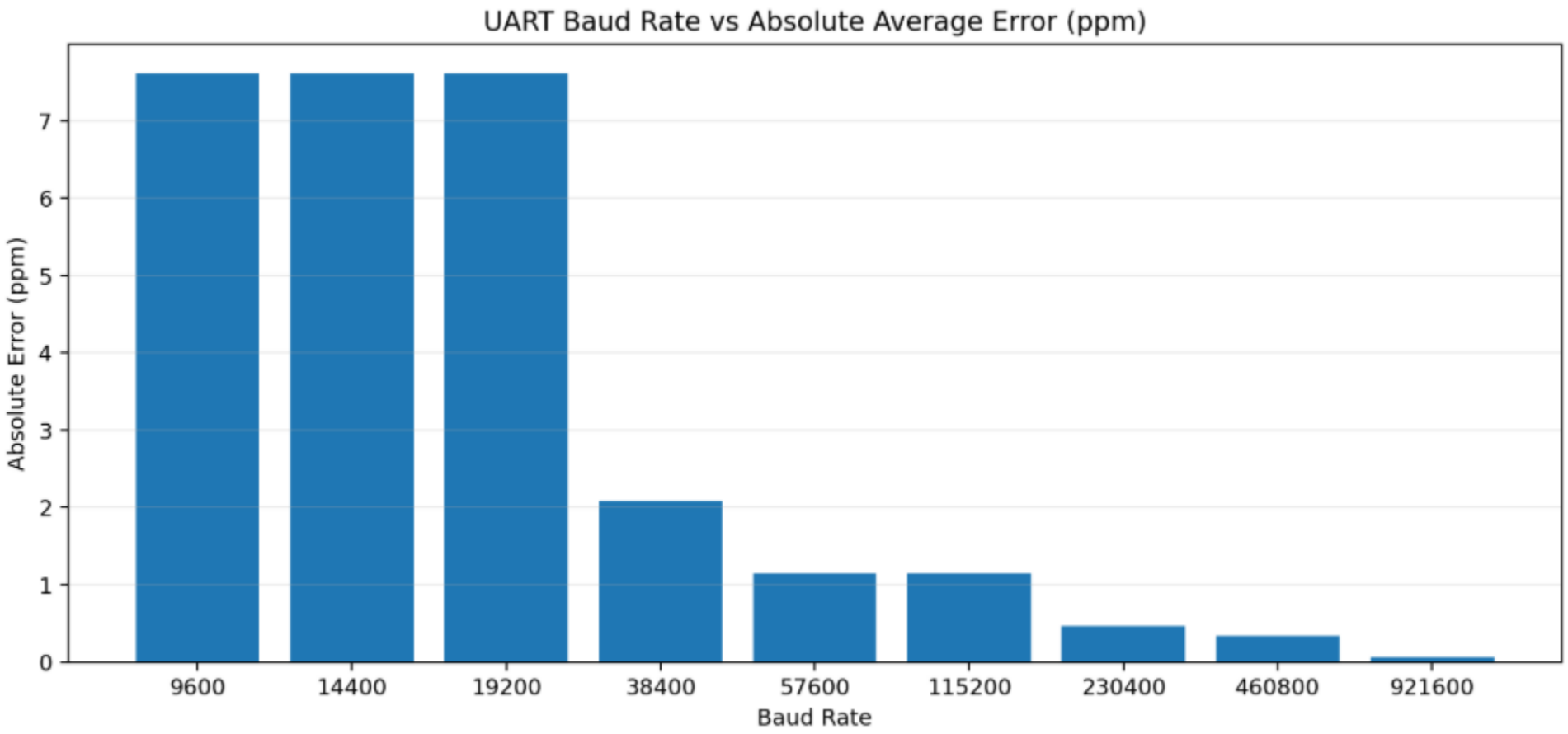
Simulation

시나리오	stimulous	기대 결과	주요 검증 포인트
Reset / ID	reset 후 UART_ID, UART_STATUS, BAUDCFG 읽기	reset 상태 정상	기본 register map, default status
TX Path	APB로 0x55, 0xA3, 0x0D 전송	o_uart_tx에서 같은 값 직렬 출력	TX FIFO, serializer, stop bit
RX Normal	clean serial 0x3C 주입	RXDATA=0x3C	RX path, RX FIFO, APB read
RX Jitter	jittered 0xA6 주입	RXDATA=0xA6	RX sampling robustness
APB Baud Select	APB BAUDCFG로 baud source 전환	APB-selected baud 수신 성공	baud source mux, APB config path
Frame Error	bad stop bit frame 주입	FRAME_ERROR set/clear	stop bit check, sticky clear
RX Overflow	32byte burst + 1byte 추가	overflow set, 마지막 byte drop	FIFO full policy, drain ordering

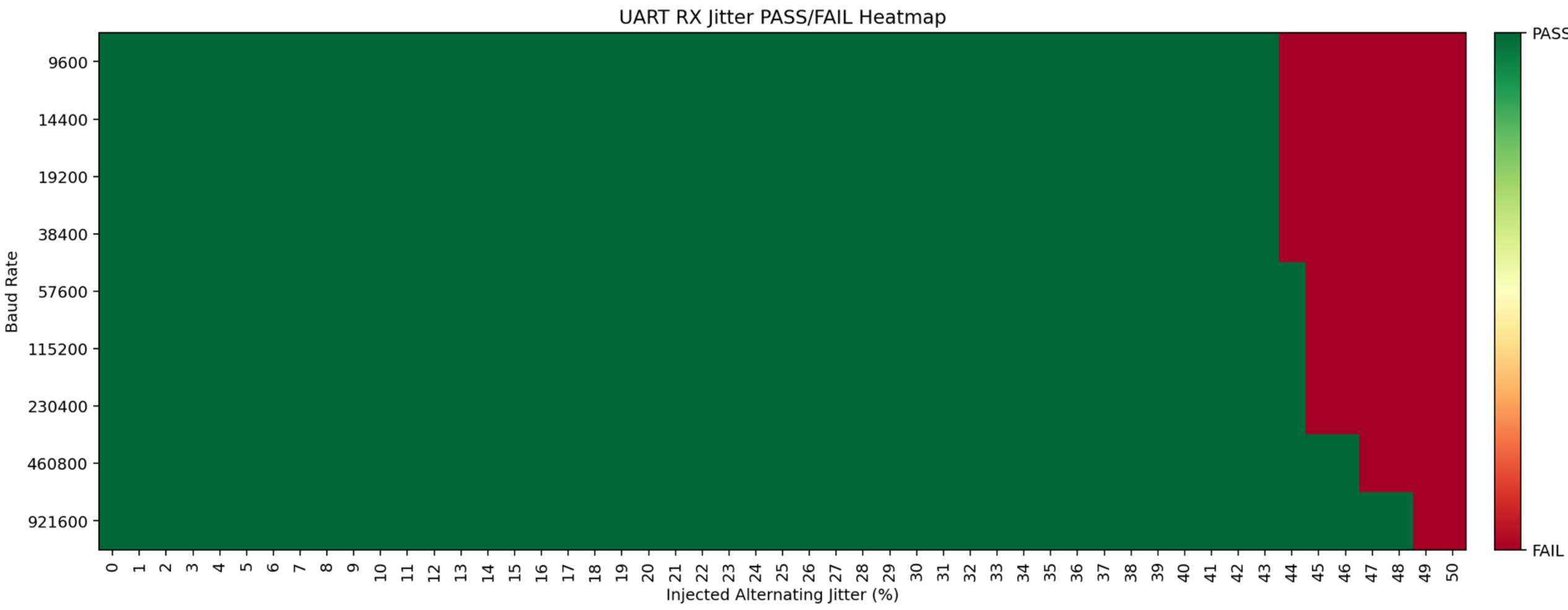
Simulation


Driver
drv_cb/mon_cb
uart_apb_wrapper
@(drv_cb) address/control setup
paddr, psel, pwrite, pwdata
@(drv_cb) penable assert
penable=1
@(mon_cb) sample prdata/pslverr
response returned
Driver
drv_cb/mon_cb
uart_apb_wrapper

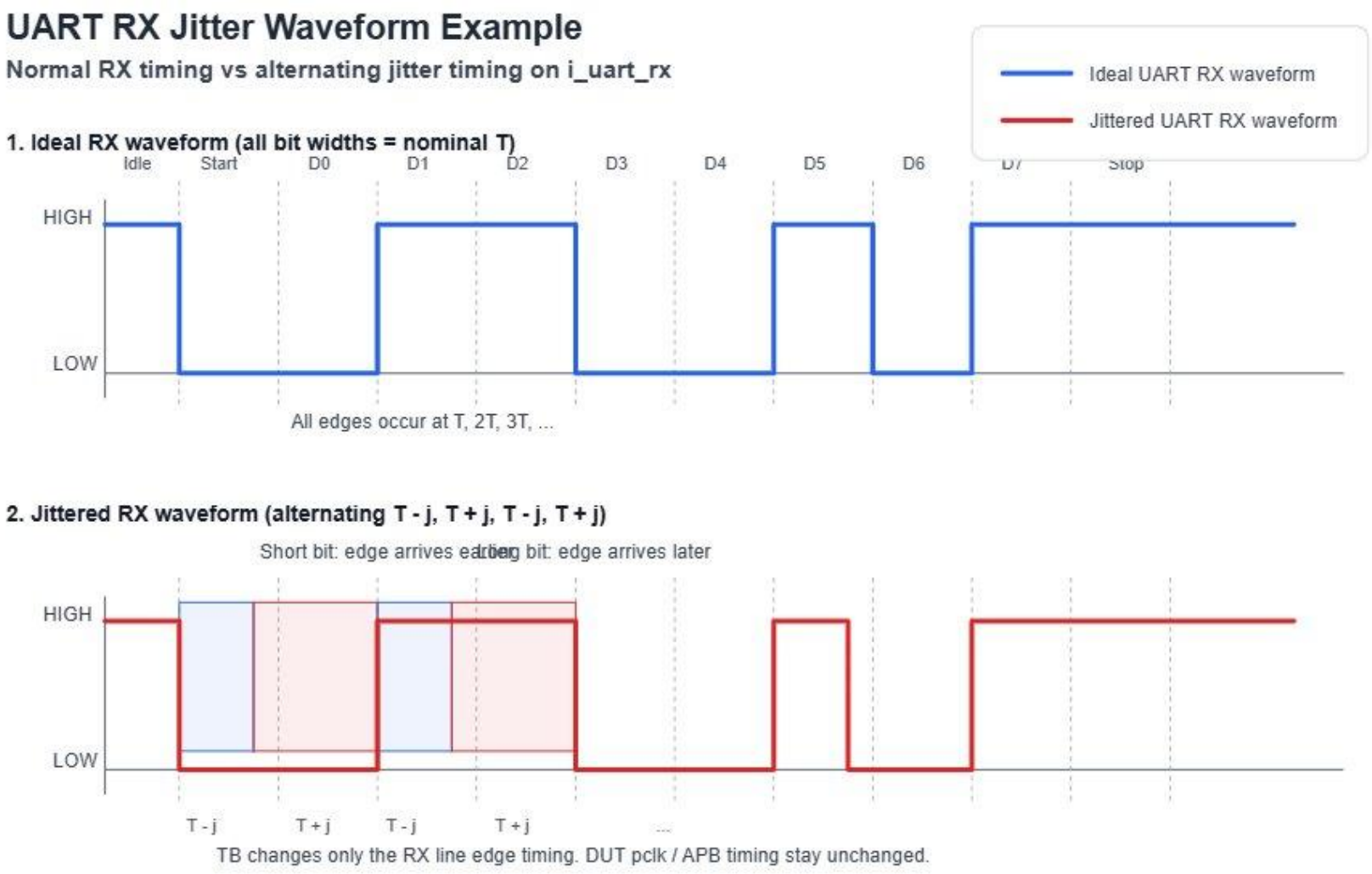
Simulation


UART Baud Rate vs Absolute Average Error (ppm)
Absolute Error (ppm)
0
1
2
3
4
5
6
7
9600
14400
19200
38400
57600
115200
230400
460800
921600
Baud Rate

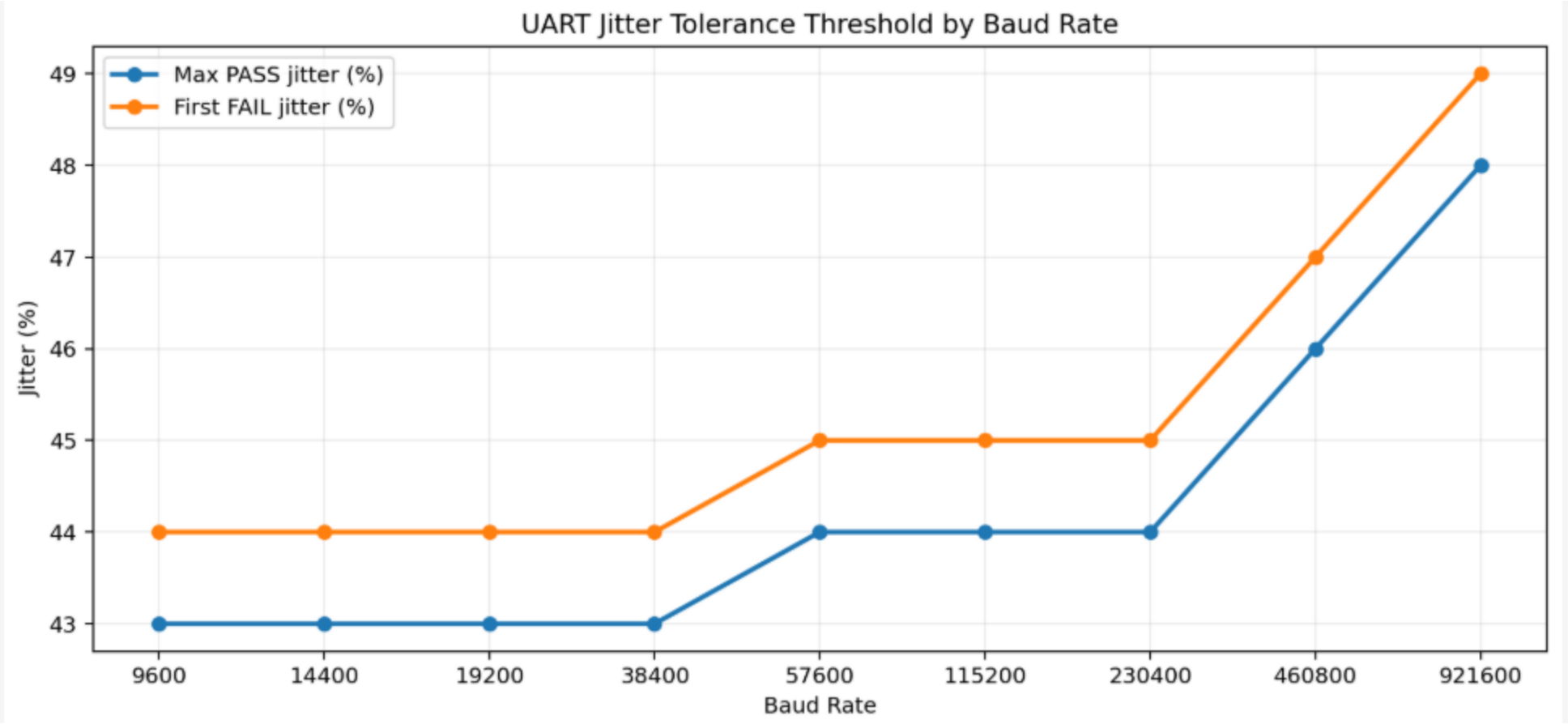
Simulation


UART RX Jitter PASS/FAIL Heatmap
Baud Rate
9600
14400
19200
38400
57600
115200
230400
460800
921600
Injected Alternating Jitter (%)
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50
PASS
FAIL

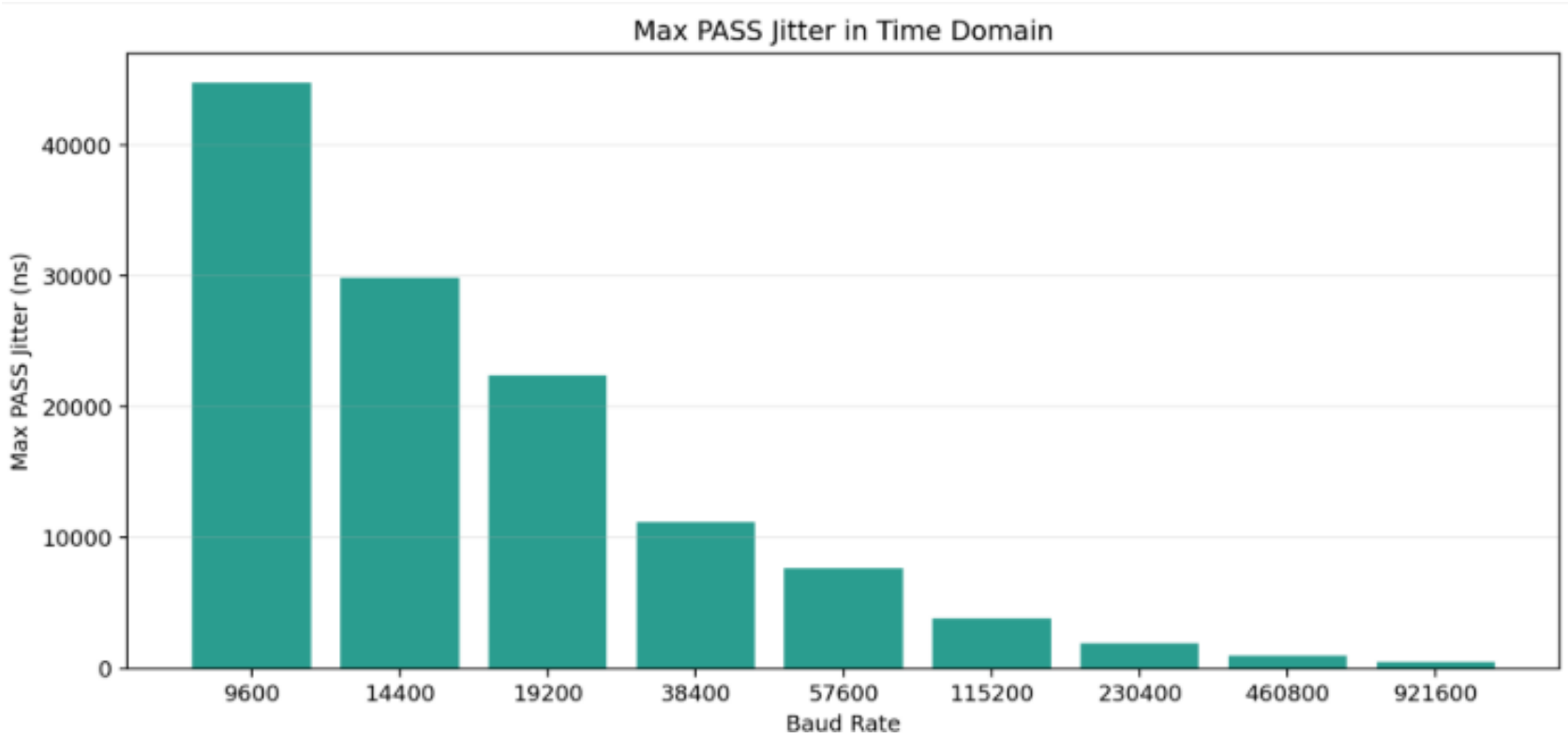
Simulation


UART RX Jitter Waveform Example
Normal RX timing vs alternating jitter timing on i_uart_rx
Ideal UART RX waveform
Jittered UART RX waveform
1. Ideal RX waveform (all bit widths = nominal T)
Idle
Start
D0
D1
D2
D3
D4
D5
D6
D7
Stop
HIGH
LOW
All edges occur at T, 2T, 3T, ...
2. Jittered RX waveform (alternating T - j, T + j, T - j, T + j)
Short bit: edge arrives earlier
Long bit: edge arrives later
HIGH
LOW
T - j
T + j
T - j
T + j
...
TB changes only the RX line edge timing. DUT pclk / APB timing stay unchanged.

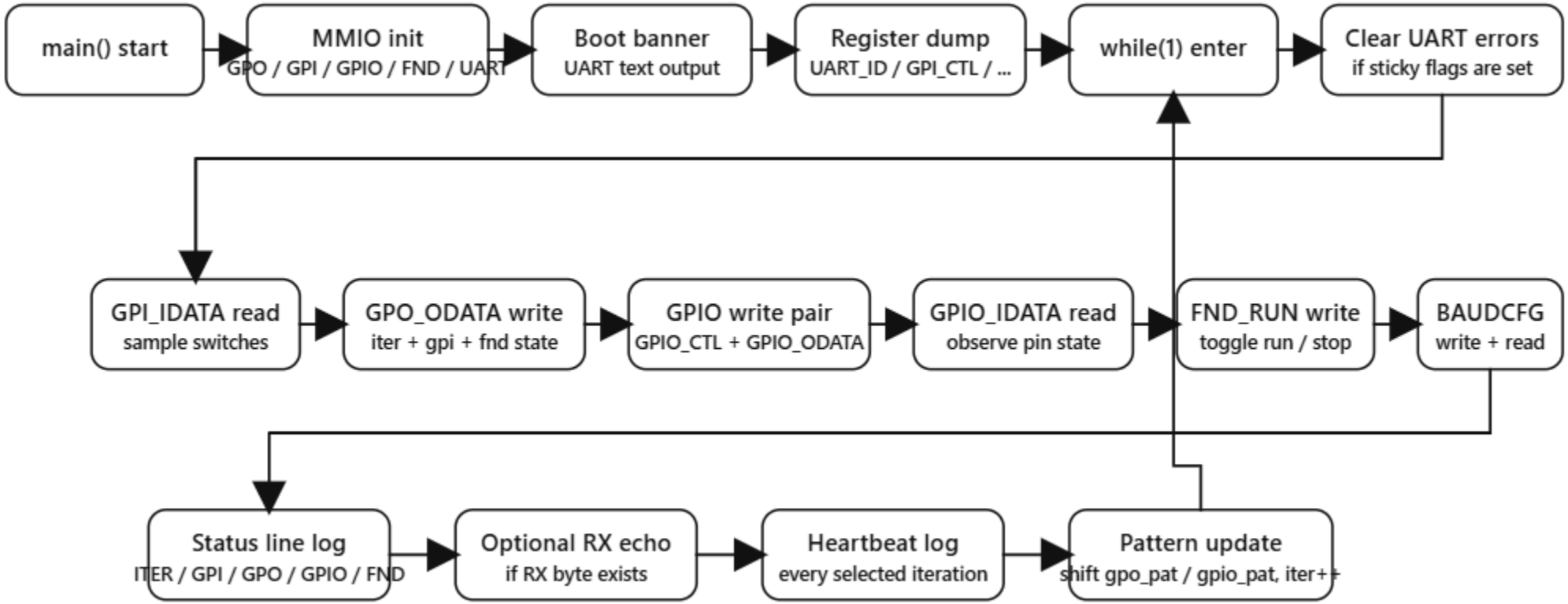
Simulation


UART Jitter Tolerance Threshold by Baud Rate
Max PASS jitter (%)
First FAIL jitter (%)
Jitter (%)
49
48
47
46
45
44
43
9600
14400
19200
38400
57600
115200
230400
460800
921600
Baud Rate

Simulation


Max PASS Jitter in Time Domain
Max PASS Jitter (ns)
40000
30000
20000
10000
0
9600
14400
19200
38400
57600
115200
230400
460800
921600
Baud Rate

TOP


main() start
MMIO init
GPO / GPI / GPIO / FND / UART
Boot banner
UART text output
Register dump
UART_ID / GPI_CTL / ...
while(1) enter
Clear UART errors
if sticky flags are set
GPI_IDATA read
sample switches
GPO_ODATA write
iter + gpi + fnd state
GPIO write pair
GPIO_CTL + GPIO_ODATA
GPIO_IDATA read
observe pin state
FND_RUN write
toggle run / stop
BAUDCFG
write + read
Status line log
ITER / GPI / GPO / GPIO / FND
Optional RX echo
if RX byte exists
Heartbeat log
every selected iteration
Pattern update
shift gpo_pat / gpio_pat, iter++

TOP

```
[195000][TOP_TB] Reset released
[1695000][PERI_MMIO] pc=0000053c WR addr=20000000 wdata=0000ffff rdata=00000000 err=0
[1835000][PERI_MMIO] pc=00000544 WR addr=20000004 wdata=00000000 rdata=00000000 err=0
[2035000][PERI_MMIO] pc=00000550 WR addr=20001000 wdata=000007ff rdata=00000000 err=0
[2235000][PERI_MMIO] pc=0000055c WR addr=20002000 wdata=0000000f rdata=00000000 err=0
[2375000][PERI_MMIO] pc=00000564 WR addr=20002004 wdata=00000000 rdata=00000000 err=0
[2575000][PERI_MMIO] pc=00000570 WR addr=20003000 wdata=00000001 rdata=00000000 err=0
[2775000][PERI_MMIO] pc=0000057c WR addr=20004010 wdata=00000003 rdata=00000000 err=0
[2915000][PERI_MMIO] pc=00000584 WR addr=20004010 wdata=00000000 rdata=00000000 err=0
[3175000][PERI_MMIO] pc=00000594 WR addr=20004014 wdata=00000015 rdata=00000505 err=0
[1740455000][TOP_UART] CPU ROM peripheral repeat test boot
[1740785000][PERI_MMIO] pc=000005b4 RD addr=20004000 wdata=00000000 rdata=55415254 err=0
[3476505000][TOP_UART] UART_ID=0x55415254
[3476785000][PERI_MMIO] pc=000005c4 RD addr=20001000 wdata=00000000 rdata=000007ff err=0
[5212595000][TOP_UART] GPI_CTL=0x000007FF
[5212875000][PERI_MMIO] pc=000005d4 RD addr=20000000 wdata=00000000 rdata=0000ffff err=0
[6948775000][TOP_UART] GPO_CTL=0x0000FFFF
[6949055000][PERI_MMIO] pc=000005e4 RD addr=20002000 wdata=00000000 rdata=0000000f err=0
[8771735000][TOP_UART] GPIO_CTL=0x0000000F
[8772015000][PERI_MMIO] pc=000005f4 RD addr=20003000 wdata=00000000 rdata=00000001 err=0
[10507735000][TOP_UART] FND_RUN=0x00000001
[10508015000][PERI_MMIO] pc=00000604 RD addr=20004014 wdata=00000000 rdata=00000515 err=0
[14674355000][TOP_UART] BAUDCFG src=A req=0x05 act=0x05 rate=115200bps
```

TOP

```
[14678455000][PERI_MMIO] pc=00000700 RD addr=20001004 wdata=00000000 rdata=00000275 err=0
[14682595000][PERI_MMIO] pc=0000072c WR addr=20000004 wdata=00008275 rdata=00000000 err=0
[14686625000][PERI_MMIO] pc=00000750 WR addr=20002000 wdata=0000000f rdata=0000000f err=0
[14686715000][PERI_MMIO] pc=00000754 WR addr=20002004 wdata=00000001 rdata=00000000 err=0
[14690695000][PERI_MMIO] pc=00000774 RD addr=20002008 wdata=00000000 rdata=0000zzz0 err=0
[14694585000][PERI_MMIO] pc=00000790 WR addr=20003000 wdata=00000000 rdata=00000001 err=0
[14698955000][PERI_MMIO] pc=000007c8 WR addr=20004014 wdata=00000018 rdata=00000515 err=0
[14699045000][PERI_MMIO] pc=000007cc RD addr=20004014 wdata=00000000 rdata=00000818 err=0
[15032745000][PERI_MMIO] pc=00000890 RD addr=20000004 wdata=00000000 rdata=00008275 err=0
[15444855000][TOP_UART] ITER 0x00 GPI=0x00000275 GPO=0x00008275 GPIO_IDATA=0x0000 FND=0
[15444855000][TOP_TB] Peripheral repeat loop observed successfully
[15444855000][TOP_TB] first_pc GPI_IDATA    = 00000700
[15444855000][TOP_TB] first_pc GPO_ODATA    = 00000544
[15444855000][TOP_TB] first_pc GPIO_CTL     = 0000055c
[15444855000][TOP_TB] first_pc GPIO_ODATA   = 00000564
[15444855000][TOP_TB] first_pc GPIO_IDATA   = 00000774
[15444855000][TOP_TB] first_pc FND_RUN      = 00000570
[15444855000][TOP_TB] first_pc UART_BAUDCFG_WR = 00000594
[15444855000][TOP_TB] first_pc UART_BAUDCFG_RD = 00000604
[15444855000][TOP_TB] first_pc UART_TXDATA  = 0000003c
[15444855000][TOP_TB] uart_line_count=8 iter_line_count=1 gpo=8275 gpio=xxxx fnd_an=1101 seg=1000000
```

Consideration

나연우

- 팀원들과 자료를 공유하면서 데이터의 정합성과 호환성을 만족시키는 것이 굉장히 어려웠습니다.
- 이를 통해 팀원들과 협업하는 것은 단순히 ' 하는 ' 것이 아니라 기술이라는 것을 새삼 이해하게 되었습니다.

최여지

- 고속의 메인 시스템과 저속 주변 장치 간의 통신에서 APB 브리지의 역할이 얼마나 중요한지 이해할 수 있었습니다.
- 하드웨어 수준의 주소 해독(Address Decoding)과 상태 머신(FSM) 타이밍을 설계하고 시뮬레이션으로 검증해 보며, 버스 충돌과 데이터 유실을 막는 인터페이스 설계의 필요성에 대해 알게 되었습니다.

우상욱

- STM32의 데이터시트로만 봤던 하드웨어를 비슷하게 구현해보면서 이해도를 높일 수 있었던 프로젝트였습니다.
- 성능을 올리기 위해서는 AXI, AHB, DMA와 Pipeline 구조를 도입의 필요성을 알게 되었습니다.